

# 燕 山 大 学

## 误差理论与数据处理

### 项目作业报告

#### 22 级检测技术与仪器专业

|                      |      |      |      |        |
|----------------------|------|------|------|--------|
| 作业题目：伏安法测电阻及其数据处理    |      |      |      |        |
| 所属班级：检测 22-2 班       |      |      |      |        |
| 学生姓名：                | 游寅龙  | 黄林烁  | 韩昱享  | 陈旭洋    |
| 任务分工：                | 代码编写 | 公式整理 | 撰写报告 | PPT 制作 |
| 贡献度：                 | 40%  | 20%  | 20%  | 20%    |
| 指导教师：李昕              |      |      |      |        |
| 完成日期：2024 年 5 月 28 日 |      |      |      |        |

# 一、引言

本文通过误差理论与数据处理的相关手段，对测得电压和电流进行了统计分析，并使用最小二乘法作出了对电阻值的估计。

# 二、研究内容

研究大致基于以下内容展开：

1. 测量了两组不同精度下的同一电压值，对每组数据做等精度处理后，对两次统计结果作不等精度数据处理，得出最终结果。
2. 测量了两组不同精度下的同一电流值，对每组数据做等精度处理后，对两次统计结果作不等精度数据处理，得出最终结果。
3. 测量了一组同精度下的不同一电压值，及对应等进度电流值，通过最小二乘法的手段进行求值运算，得出未知电阻  $R_x$  及电压表电流  $I_u$  的估算值。

# 三、研究流程

研究大致分为四个步骤，其研究流程如图 3-1：

1. 设计实验，绘制对应测试电路图，得到实验电路板。
2. 根据实验预先设计，进行数据测量。
3. 不改变电位器电阻值下，等精度测量各两组电压和电流值，等进度处理后，作不等精度处理。
4. 改变电位器电阻后，测量多组电压电流的对应值，并进行最小二乘法处理。

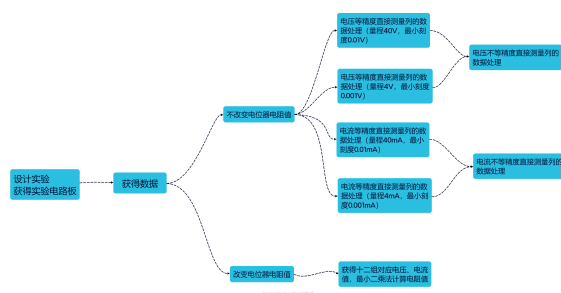


图 3-1: 实验流程图

## (一) 设计实验

经过小组讨论后，最终确认本次实验围绕电学物理实验——“伏安法测电阻”进行。

伏安法（又称伏特测量法、安培测量法）是一种较为普遍的测量电阻的方法，通过利用部分电路欧姆定律： $R = \frac{U}{I}$  来测出电阻值。用电流表测出在此电压下通过未知电阻的电流，然后计算出未知电阻的阻值，这种测电阻的方法，叫伏安法。

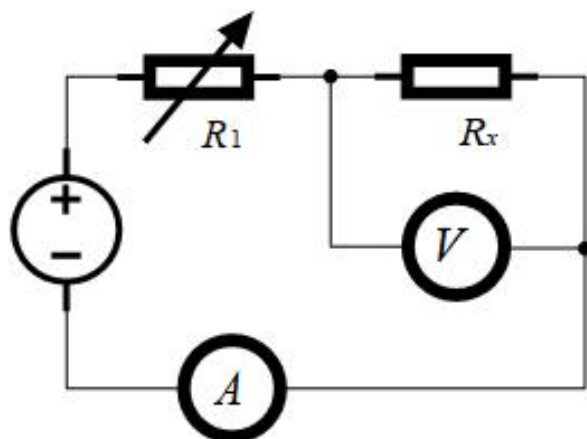
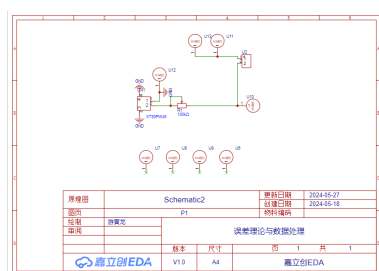


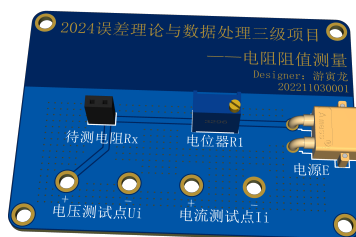
图 3-2: 伏安法测电阻实验原理图

基于伏安法的实验原理，可以得到实验原理图（如图 3-2）。本次实验，采用电流表外接的形式。因此需要在后续进行数据处理时考虑电压表内阻通过的电流，对实验的影响。

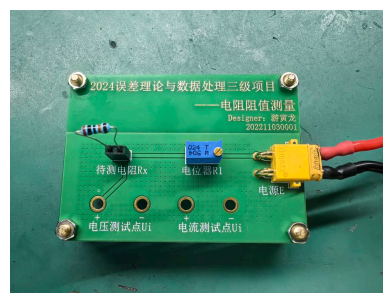
根据实验原理图，便可以设计实际原理图和电路板。



(a) 实验电路板原理图



(b) 实验电路板仿真图



(c) 实验电路板实际图

图 3-3: 伏安法测电阻实验电路板

## (二) 数据测量

测量使用两个同样型号（型号：LCSC530+）的万用表进行，实际测量方式如图 3-4。其中，左侧表作为电压表使用，测量电压；右侧表作为电流表使用，测量电流。

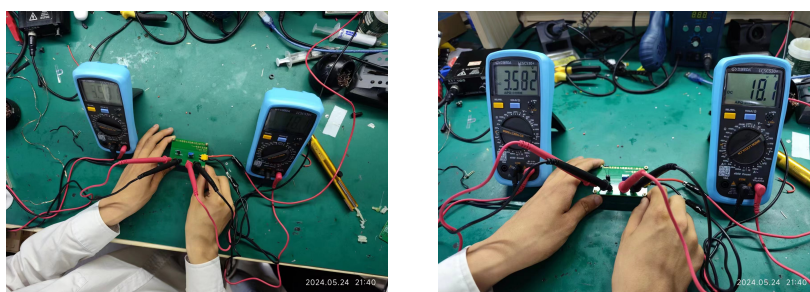


图 3-4: 伏安法测电阻实验测量方式

首先进行等精度下的电压测量。

使用量程为 40V，最小分度值为 0.01V 的档位，测量电压数据如表 3-1所示。

表 3-1: 量程为 40V、最小分度值为 0.01V 时电压采样数据

| 次数      | 1    | 2    | 3    | 4    | 5    | 6    |
|---------|------|------|------|------|------|------|
| 测量值 (V) | 3.57 | 3.58 | 3.58 | 3.58 | 3.58 | 3.57 |

表 3-2: 量程为 4V、最小分度值为 0.001V 时电压采样数据

| 次数      | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     |
|---------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 测量值 (V) | 3.578 | 3.580 | 3.582 | 3.580 | 3.577 | 3.583 | 3.586 | 3.579 | 3.584 |

使用量程为 4V，最小分度值为 0.001V 的档位，测量电压数据如表 3-2所示。

随后进行等精度下的电流测量。

使用量程为 400 $\mu$ A，最小分度值为 0.1 $\mu$ A 的档位，测量电流数据如表 3-3所示。

表 3-3: 量程为 400 $\mu$ A，最小分度值为 0.1 $\mu$ A 时电流采样数据

| 次数             | 1    | 2    | 3    | 4    | 5    | 6    |
|----------------|------|------|------|------|------|------|
| 测量值 ( $\mu$ A) | 18.0 | 18.1 | 18.1 | 18.0 | 18.1 | 18.0 |

使用量程为 40 $\mu$ A，最小分度值为 0.01 $\mu$ A 的档位，测量电流数据如表 3-4所示。

表 3-4: 量程为 40 $\mu$ A，最小分度值为 0.01 $\mu$ A 时电流采样数据

| 次数             | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     |
|----------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 测量值 ( $\mu$ A) | 18.09 | 18.09 | 18.09 | 18.08 | 18.09 | 18.09 | 18.08 | 18.09 | 18.08 |

最后对电位器电阻进行改变，测得多组电压电流的对应值。

其中，电压档选用量程为 4V、最小分度值为 0.001V，电流档选用量程为 40 $\mu$ A，最小分度值为 0.01 $\mu$ A。测量结果如表 3-5所示。

表 3-5: 量程为 40 $\mu$ A，最小分度值为 0.01 $\mu$ A 时电流采样数据

| 次数               | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     |
|------------------|-------|-------|-------|-------|-------|-------|-------|-------|
| 电压测量值 (V)        | 0.530 | 0.569 | 1.013 | 1.822 | 1.824 | 2.058 | 2.557 | 2.701 |
| 电流测量值 ( $\mu$ A) | 2.83  | 3.01  | 5.25  | 9.33  | 9.36  | 10.49 | 13.00 | 13.73 |
| 次数               | 9     | 10    | 11    | 12    | 13    | 14    | 15    | 16    |
| 电压测量值 (V)        | 3.039 | 3.920 | 4.610 | 4.765 | 4.811 | 5.644 | 6.181 | 6.186 |
| 电流测量值 ( $\mu$ A) | 15.42 | 19.86 | 23.34 | 24.12 | 24.34 | 28.54 | 31.24 | 31.29 |

### (三) 数据处理

首先针对表 3-1、表 3-2、表 3-3、表 3-4中的四次数据进行等精度处理。

随后针对表 3-1和表 3-2等精度处理后的电压值、表 3-3、表 3-4等精度处理后的电流值，进行不等精度数据处理。

最后针对表 3-5中的数据进行最小二乘法估算电阻值。

其详细过程将在后续陈述。

## 四、 理论原理

以下, 对于《误差理论与数据处理》一书中所述等精度数据处理方法做出简要叙述。

### (一) 等精度数据处理

等精度测量数据处理是指在整个测量过程中, 影响和决定误差大小的全部因素(条件)始终保持不变。

等精度测量的数据处理分为九个步骤: 1. 求算术平均值 2. 求残余误差 3. 校核算术平均值及其残余误差 4. 判断系统误差 5. 求测量列单次测量的标准差 6. 判断粗大误差 7. 求算术平均值的标准差 8. 求算术平均值的极限误差 9. 写出最后测量结果。

定义: 等精度测量列为  $l_i$ 。

#### 1. 求算术平均值

算术平均值指被测量的  $n$  个测得值的总和除以  $n$  而得的值。根据定义, 算术平均值  $\bar{x}$  的计算方法为:

$$\bar{x} = \frac{l_1 + l_2 + \cdots + l_n}{n} = \frac{\sum_{i=1}^n l_i}{n} \quad (1)$$

#### 2. 求残余误差

残余误差指测得值与真值之间的差值。由于真值一般无法得到, 因此使用最接近真值的算术平均值  $\bar{x}$  代替真值。据此残余误差  $v_i$  的计算方法为:

$$v_i = l_i - \bar{x} \quad i = 1, 2, \cdots, n$$

#### 3. 校核算术平均值及其残余误差

算术平均值及其残余误差的计算是否正确, 可用求得的残余误差代数和的性质来校核。其有两个可行的规则。

规则一 残余误差代数和应满足:

- 当  $\sum_{i=1}^n l_i = n\bar{x}$ , 求得的  $\bar{x}$  为非凑整的准确数时,  $\sum_{i=1}^n v_i$  为零
- 当  $\sum_{i=1}^n l_i > n\bar{x}$ , 求得的  $\bar{x}$  为凑整的非准确数时,  $\sum_{i=1}^n v_i$  为正, 其大小为求  $\bar{x}$  的余数
- 当  $\sum_{i=1}^n l_i < n\bar{x}$ , 求得的  $\bar{x}$  为凑整的非准确数时,  $\sum_{i=1}^n v_i$  为负, 其大小为求  $\bar{x}$  的亏数

规则二 残余误差代数和绝对值应符合:

- 当  $n$  为偶数时,  $\left| \sum_{i=1}^n v_i \right| \leq \frac{n}{2} A$
- 当  $n$  为奇数时,  $\left| \sum_{i=1}^n v_i \right| \leq (\frac{n}{2} - 0.5) A$

式中的  $A$  为实际求得的算术平均值  $\bar{x}$  末位数的一个单位。

## 4. 判断系统误差

判断等精度测量系统误差共具有四种方法：1. 实验对比法 2. 残余误差观察法 3. 残余误差校核法 4. 不同公式计算标准差比较法。由于前两种方法过于主观，不具有计算可行性，故只说明后续两种方法。

**残余误差校核法 (马利科夫准则)** 将测量列中前  $K$  个残余误差相加，后  $(n - K)$  个残余误差相加，两者相减。当  $n$  为偶数，取  $K = \frac{n}{2}$ ；当  $n$  为奇数，取  $K = \frac{n+1}{2}$ 。公式表达如下：

$$\Delta = \sum_{i=1}^K v_i - \sum_{j=K+1}^n v_j \approx \sum_{i=1}^K (l_i - \Delta \bar{x}) - \sum_{j=K+1}^n (l_j - \Delta \bar{x})$$

如果两部分的差值  $\Delta$  显著不为零，则可认为测量列存在线性系统误差。

**不同公式计算标准差比较法** 本方法需要使用式 2 的贝塞尔公式所算出的标准差  $\sigma$  和式 3 的别捷尔斯公式所算出的标准差  $\sigma'$ 。

令

$$\frac{\sigma'}{\sigma} = 1 + u$$

若

$$|u| \geq \frac{2}{\sqrt{n-1}}$$

则怀疑测量列中存在系统误差。

## 5. 求测量列单次测量的标准差

测量列的标准偏差简称为标准差，也称之为方均根误差。

求测量列单次测量的标准差共具有四种方法：1. 贝塞尔公式 2. 别捷尔斯法 3. 极差法 4. 最大误差法。

由于最大误差法精度较低，不适用于本实验，便没有采用。

**贝塞尔公式**

$$\sigma = \sqrt{\frac{\sum_{i=1}^n v_i^2}{n-1}} \quad (2)$$

**别捷尔斯法**

$$\sigma' = 1.253 \frac{\sum_{i=1}^n |v_i|}{\sqrt{n(n-1)}} \quad (3)$$

**极差法** 若等精度多次测量测得值  $l_1, l_2, \dots, l_n$  服从正态分布，在其中选取最大值  $l_{max}$  与最小值  $l_{min}$ ，则两者之间的差称为极差，即

$$\omega_n = l_{max} - l_{min}$$

标准差  $\sigma$  计算方式为

$$\sigma = \frac{\omega_n}{d_n}$$

式中， $d_n$  的数值见表 4-1

表 4-1: 极差系数表

|   |      |      |      |      |      |      |      |      |      |      |
|---|------|------|------|------|------|------|------|------|------|------|
| n | /    | 2    | 3    | 4    | 5    | 6    | 7    | 8    | 9    | 10   |
| d | /    | 1.13 | 1.69 | 2.06 | 2.33 | 2.53 | 2.70 | 2.85 | 2.97 | 3.08 |
| n | 11   | 12   | 13   | 14   | 15   | 16   | 17   | 18   | 19   | 20   |
| d | 3.17 | 3.26 | 3.34 | 3.41 | 3.47 | 3.53 | 3.59 | 3.64 | 3.69 | 3.74 |

## 6. 判断粗大误差

粗大误差的数值比较大，会对测量结果产生明显的歪曲，一旦发现含有粗大误差的测量值，应将其从测量结果中剔除。

判断粗大误差共具有四种方法：1.  $3\sigma$  准则（莱以特准则）2. 罗曼诺夫斯基准则 3. 格罗布斯准则 4. 狄克松准则。

由于狄克松准则适用于测量次数少时的快速判断，不适用于本实验，便没有采用。

**$3\sigma$  准则（莱以特准则）** 若

$$|v_i| > 3\sigma$$

则可以认为它含有粗大误差，应当予以剔除。

**罗曼诺夫斯基准则** 若认为测量值  $l_j$  为可疑数据，计算剔除可疑数据后的算术平均值和测量列的标准差

$$\begin{aligned}\bar{x}' &= \frac{1}{n-1} \sum_{i=1, i \neq j}^n l_i \\ v'_i &= l_i - \bar{x}' \quad i = 1, 2, \dots, j-1, j+1, \dots, n \\ \sigma'' &= \sqrt{\frac{\sum_{i=1}^n v_i'^2}{n-2}}\end{aligned}$$

根据测量次数  $n$  和选取的显著度  $\alpha$ ，可由表 4-2 查得  $t$  分布的检验系数  $K(n, \alpha)$ 。

若

$$|l_j - \bar{x}'| > K\sigma''$$

则认为测量值  $l_j$  含有粗大误差，剔除  $l_j$  是正确的。

表 4-2: 罗曼诺夫斯基系数表

| $K \backslash \alpha$<br>$n$ | 0.05 | 0.01  | $K \backslash \alpha$<br>$n$ | 0.05 | 0.01 | $K \backslash \alpha$<br>$n$ | 0.05 | 0.01 |
|------------------------------|------|-------|------------------------------|------|------|------------------------------|------|------|
| 4                            | 4.97 | 11.46 | 13                           | 2.29 | 3.23 | 22                           | 2.14 | 2.91 |
| 5                            | 3.56 | 6.53  | 14                           | 2.26 | 3.17 | 23                           | 2.13 | 2.90 |
| 6                            | 3.04 | 5.04  | 15                           | 2.24 | 3.12 | 24                           | 2.12 | 2.88 |
| 7                            | 2.78 | 4.36  | 16                           | 2.22 | 3.08 | 25                           | 2.11 | 2.86 |
| 8                            | 2.62 | 3.96  | 17                           | 2.20 | 3.04 | 26                           | 2.10 | 2.85 |
| 9                            | 2.51 | 3.71  | 18                           | 2.18 | 3.01 | 27                           | 2.10 | 2.84 |
| 10                           | 2.43 | 3.54  | 19                           | 2.17 | 3.00 | 28                           | 2.09 | 2.83 |
| 11                           | 2.37 | 3.41  | 20                           | 2.16 | 2.95 | 29                           | 2.09 | 2.82 |
| 12                           | 2.33 | 3.31  | 21                           | 2.15 | 2.93 | 30                           | 2.08 | 2.81 |

**格罗布斯准则** 根据式 1所算的算术平均值和式 2的贝塞尔公式所算的标准差  $\sigma$ 。

若认为  $l_{max}$  为可疑值, 则计算

$$g_{max} = \frac{\bar{x} - l_{max}}{\sigma}$$

若认为  $l_{min}$  为可疑值, 则计算

$$g_{min} = \frac{\bar{x} - l_{min}}{\sigma}$$

当

$$g_i \geq g_0(n, \alpha)$$

即判别该测得值含有粗大误差, 应当予以剔除。其中  $g_0(n, \alpha)$  根据测量次数  $n$  和选取的显著度  $\alpha$ , 可由表 4-3查得。

表 4-3: 格罗布斯系数表

| n | $\alpha$         |      | n  | $\alpha$         |      | n  | $\alpha$         |      | n   | $\alpha$         |      |
|---|------------------|------|----|------------------|------|----|------------------|------|-----|------------------|------|
|   | 0.05             | 0.01 |    | 0.05             | 0.01 |    | 0.05             | 0.01 |     | 0.05             | 0.01 |
|   | $g_0(n, \alpha)$ |      |    | $g_0(n, \alpha)$ |      |    | $g_0(n, \alpha)$ |      |     | $g_0(n, \alpha)$ |      |
| 3 | 1.15             | 1.16 | 10 | 2.18             | 2.41 | 17 | 2.48             | 2.78 | 24  | 2.64             | 2.99 |
| 4 | 1.46             | 1.49 | 11 | 2.23             | 2.48 | 18 | 2.50             | 2.82 | 25  | 2.66             | 3.01 |
| 5 | 1.67             | 1.57 | 12 | 2.28             | 2.55 | 19 | 2.53             | 2.85 | 30  | 2.74             | 3.10 |
| 6 | 1.82             | 1.94 | 13 | 2.33             | 2.61 | 20 | 2.56             | 2.88 | 35  | 2.81             | 3.18 |
| 7 | 1.94             | 2.10 | 14 | 2.37             | 2.66 | 21 | 2.58             | 2.91 | 40  | 2.87             | 3.24 |
| 8 | 2.03             | 2.22 | 15 | 2.41             | 2.70 | 22 | 2.60             | 2.94 | 50  | 2.96             | 3.34 |
| 9 | 2.11             | 2.32 | 16 | 2.44             | 2.75 | 23 | 2.62             | 2.96 | 100 | 3,17             | 3.59 |

## 7. 求算术平均值的标准差

算术平均值的标准差  $\sigma_{\bar{x}}$  是表征同一被测量的各个独立测量列算术平均值分散性的参数。其计算公式为：

$$\sigma_{\bar{x}} = \frac{\sigma}{\sqrt{n}}$$

## 8. 求算术平均值的极限误差

极限误差  $\delta_{lim}\bar{x}$  的表达式为：

$$\delta_{lim}\bar{x} = \pm t_{\alpha}\sigma_{\bar{x}}$$

式中  $t_{\alpha}$  的取值方法有三种：

1. 一般直接取  $t_{\alpha} = 3$
2. 若测量次数  $n < 10$ ，则按  $t$  分布计算。
3. 若测量次数  $n \geq 10$ ，则按正态分布计算。

## 9. 写出最后测量结果

最后测量结果通常用算术平均值及其极限误差来表示，即

$$L = \bar{x} \pm \delta_{lim}\bar{x}$$

## (二) 不等精度数据处理

在科学研究或高精度测量中，往往在不同的测量条件下，用不同的仪器、不同的测量方法、不同的测量次数以及不同的测量者进行测量与对比，这种测量称为不等精度测量。

在一般测量工作中，常遇到的不等精度测量有两种情况：1. 用不同测量次数进行对比测量 2. 用不同精度的仪器进行对比测量。因为已经经过上述等精度的数据处理，因此可以确定两次观测结果不存在系统误差和粗大误差。可按下列步骤求解

1. 求加权算术平均值
2. 求残余误差
3. 求加权算术平均值的标准差
4. 求加权算术平均值的极限误差
5. 写出最后测量结果

定义：第一组等精度测量结果为  $L_1 = x_1 \pm \sigma_1$ ；第二组等精度测量结果为  $L_2 = x_2 \pm \sigma_2$

### 1. 求加权算术平均值

权代表各测量结果的可靠程度，记为  $p$ 。

由于每组测量结果的权与其相应的标准差平方成反比，可以得到权的确定公式：

$$p_1 : p_2 = \frac{1}{\sigma_1^2} : \frac{1}{\sigma_2^2}$$

再根据计算所得的权，得到加权算术平均值，其计算表达式为：

$$\bar{x} = \frac{x_1 p_1 + x_2 p_2}{p_1 + p_2}$$

## 2. 求残余误差

残余误差  $v_i$  的计算方法为:

$$v_i = x_i - \bar{x} \quad i = 1, 2$$

## 3. 求加权算术平均值的标准差

加权算术平均值的标准差的计算表达式为:

$$\sigma_{\bar{x}} = \sigma_i \sqrt{\frac{\frac{p_i}{2}}{\sum_{i=1}^m p_i}}$$

必须指出, 只有当组数  $m$  足够多时, 才能得到较为精确的加权算术平均值的标准差, 一般情况下的组数较少, 只能得到近似的估计值。

## 4. 求加权算术平均值的极限误差

测量列算术平均值的极限误差表达式为:

$$\delta_{lim} \bar{x} = \pm t \sigma_{\bar{x}}$$

其中置信系数取  $t = 3$

## 5. 写出最后测量结果

最后测量结果通常用算术平均值及其极限误差来表示, 即

$$L = \bar{x} \pm \delta_{lim} \bar{x}$$

## (三) 最小二乘法估算

最小二乘法是一种在多学科领域中获得广泛应用的数据处理方法, 可用于解决参数的最可信赖值估计、组合测量的数据处理、用实验方法来拟合经验公式以及回归分析等一系列数据处理问题。由于测量数据通常由线性测量方程组或非线性测量方程组得到, 因此将分别讨论上述两种测量情况下参数的最小二乘法处理。按照处理的具体方法不同, 可以将最小二乘法区别为代数法和矩阵法。

### 1. 代数法

列出估计量方程 根据线性测量的测量方程可以得到相应的估计量方程组

$$\begin{cases} y_1 = a_{11}x_1 + a_{12}x_2 + \cdots + a_{1t}x_t \\ y_2 = a_{21}x_1 + a_{22}x_2 + \cdots + a_{2t}x_t \\ \vdots \quad \quad \quad \vdots \quad \quad \quad \ddots \quad \quad \quad \vdots \\ y_n = a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nt}x_t \end{cases} \quad (4)$$

**列出误差方程** 根据线性测量的误差方程式一般格式（式 4），可列出相应的误差方程式。

$$\begin{cases} v_1 = l_1 - (a_{11}x_1 + a_{12}x_2 + \cdots + a_{1t}x_t) \\ v_2 = l_2 - (a_{21}x_1 + a_{22}x_2 + \cdots + a_{2t}x_t) \\ \vdots \quad \vdots \quad \quad \quad \vdots \quad \quad \quad \ddots \quad \quad \quad \vdots \\ v_n = l_n - (a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nt}x_t) \end{cases}$$

**列出正规方程** 为了获得更可靠的结果，测量次数  $n$  总要多于未知参数的个数  $t$ ，即所得误差方程式的个数总是要多于未知数的个数。因而直接用一般解代数方程的方法是无法求解这些未知参数的。最小二乘法则可以由误差方程得到有确定解的代数方程组（其方程式个数正好等于未知数的个数），从而可求解出这些未知参数。这个有确定解的代数方程组称为最小二乘法估计的正规方程（或称为法方程）。根据等精度测量的线性方程最小二乘法处理的正规方程一般格式（如），可列出相应的正规方程。

$$\begin{cases} \sum_{i=1}^n a_{i1}a_{i1}x_1 + \sum_{i=1}^n a_{i1}a_{i2}x_2 + \cdots + \sum_{i=1}^n a_{i1}a_{it}x_t = \sum_{i=1}^n a_{i1}l_i \\ \sum_{i=1}^n a_{i2}a_{i1}x_1 + \sum_{i=1}^n a_{i2}a_{i2}x_2 + \cdots + \sum_{i=1}^n a_{i2}a_{it}x_t = \sum_{i=1}^n a_{i2}l_i \\ \vdots \quad \quad \quad \vdots \quad \quad \quad \ddots \quad \quad \quad \vdots \quad \quad \quad \vdots \\ \sum_{i=1}^n a_{it}a_{i1}x_1 + \sum_{i=1}^n a_{it}a_{i2}x_2 + \cdots + \sum_{i=1}^n a_{it}a_{it}x_t = \sum_{i=1}^n a_{it}l_i \end{cases} \quad (5)$$

该方程组在形式上有如下特点：

1. 沿主对角线分布着平方项系数都为整数。
2. 以主对角线为对称线，对称分布的各系数彼此两两相等。

**求解估计量** 求解方程组 5，得到对应的  $x_i$

**计算测量数据的标准差** 对测量数据最小二乘法处理的最终结果，不仅要给出待求量的最可信赖的估计量，而且还要确定其可信赖程度，即应给出所得估计量的精度。

为了确定最小二乘估计量  $x_1, x_2, \cdots, x_t$  的精度，首先需要给出直接测量所得测量数据的精度。测量数据的精度也以标准差来表示。

将已求得的估计量数值代入  $x_i$  标准差公式

$$\sigma = \sqrt{\frac{\sum_{i=1}^n v_i^2}{n - t}}$$

式中  $n$  为直接测量的次数， $t$  为未知量的个数。

**计算各估计量的标准差** 在正规方程组中，利用不定乘数法求出  $x_1, x_2, \cdots, x_t$  的表达式，然后再找出估计量  $x_1, x_2, \cdots, x_t$  的精度与测量数据  $l_1, l_2, \cdots, l_n$  精度的关系，即可得到估计量精度估计的表达式。

设有不定乘数  $d_{11}, d_{12}, \cdots, d_{1t} \quad d_{21}, d_{22}, \cdots, d_{2t} \quad \cdots \quad d_{t1}, d_{t2}, \cdots, d_{tt}$  分别为下列各方程组的解:

$$\left\{ \begin{array}{l} \sum_{i=1}^n a_{i1}a_{i1}d_1 + \sum_{i=1}^n a_{i1}a_{i2}d_2 + \cdots + \sum_{i=1}^n a_{i1}a_{it}d_t = 1 \\ \sum_{i=1}^n a_{i2}a_{i1}d_1 + \sum_{i=1}^n a_{i2}a_{i2}d_2 + \cdots + \sum_{i=1}^n a_{i2}a_{it}d_t = 0 \\ \vdots \qquad \qquad \qquad \vdots \qquad \qquad \qquad \ddots \qquad \qquad \qquad \vdots \qquad \qquad \vdots \\ \sum_{i=1}^n a_{it}a_{i1}d_1 + \sum_{i=1}^n a_{it}a_{i2}d_2 + \cdots + \sum_{i=1}^n a_{it}a_{it}d_t = 0 \\ \sum_{i=1}^n a_{i1}a_{i1}d_1 + \sum_{i=1}^n a_{i1}a_{i2}d_2 + \cdots + \sum_{i=1}^n a_{i1}a_{it}d_t = 0 \\ \sum_{i=1}^n a_{i2}a_{i1}d_1 + \sum_{i=1}^n a_{i2}a_{i2}d_2 + \cdots + \sum_{i=1}^n a_{i2}a_{it}d_t = 1 \\ \vdots \qquad \qquad \qquad \vdots \qquad \qquad \qquad \ddots \qquad \qquad \qquad \vdots \qquad \qquad \vdots \\ \sum_{i=1}^n a_{it}a_{i1}d_1 + \sum_{i=1}^n a_{it}a_{i2}d_2 + \cdots + \sum_{i=1}^n a_{it}a_{it}d_t = 0 \\ \vdots \\ \sum_{i=1}^n a_{i1}a_{i1}d_1 + \sum_{i=1}^n a_{i1}a_{i2}d_2 + \cdots + \sum_{i=1}^n a_{i1}a_{it}d_t = 0 \\ \sum_{i=1}^n a_{i2}a_{i1}d_1 + \sum_{i=1}^n a_{i2}a_{i2}d_2 + \cdots + \sum_{i=1}^n a_{i2}a_{it}d_t = 0 \\ \vdots \qquad \qquad \qquad \vdots \qquad \qquad \qquad \ddots \qquad \qquad \qquad \vdots \qquad \qquad \vdots \\ \sum_{i=1}^n a_{it}a_{i1}d_1 + \sum_{i=1}^n a_{it}a_{i2}d_2 + \cdots + \sum_{i=1}^n a_{it}a_{it}d_t = 1 \end{array} \right.$$

求解上述方程组, 解得不定乘数  $d_{11}, d_{22}, \dots, d_{tt}$  的值。

其中各估计量  $x_1, x_2, \dots, x_t$  相应的标准差与测量数据的标准差有以下的对应关系:

$$\begin{cases} \sigma_{x_1} = \sigma\sqrt{d_{11}} \\ \sigma_{x_2} = \sigma\sqrt{d_{22}} \\ \vdots \\ \sigma_{x_t} = \sigma\sqrt{d_{tt}} \end{cases} \quad (6)$$

故可解得相应估计量的标准差。

## 2. 矩阵法

列出误差方程 有列向量:

$$\mathbf{L} = \begin{pmatrix} l_1 \\ l_2 \\ \vdots \\ l_n \end{pmatrix} \quad \hat{\mathbf{X}} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_t \end{pmatrix} \quad \mathbf{V} = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{pmatrix}$$

和  $n \times t$  阶矩阵 ( $n > t$ ):

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1t} \\ a_{21} & a_{22} & \cdots & a_{2t} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nt} \end{pmatrix}$$

公式中各矩阵元素：

$l_1, l_2, \dots, l_n$  为  $n$  个直接测量结果（已获得的测量数据）。

$x_1, x_2, \dots, x_t$  为  $t$  个待求的被测量的估计量。

$v_1, v_2, \dots, v_n$  为  $n$  个直接测量结果的残余误差。

$a_{11}, a_{12}, \dots, a_{1t}$  为  $n$  个误差方程的  $n \times t$  个系数。

则线性测量参数的误差方程式可表示为：

$$\mathbf{V} = \mathbf{L} - \mathbf{A}\hat{\mathbf{X}}$$

列出正规方程 正规方程的矩阵表示形式为：

$$\mathbf{A}^T \mathbf{V} = 0$$

求解估计量 令

$$\mathbf{C} = \mathbf{A}^T \mathbf{A}$$

则可得正规方程解的矩阵表达式为：

$$\hat{\mathbf{X}} = \mathbf{C}^{-1} \mathbf{A}^T \mathbf{L} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{L}$$

计算测量数据的标准差 由测量数据的标准差公式：

$$\sigma = \sqrt{\frac{\mathbf{V}^T \mathbf{V}}{n - t}}$$

可以得到，式中  $n$  为直接测量的次数， $t$  为未知量的个数。

计算估计量的标准差 做出如下定义：

$$\mathbf{C}^{-1} = \mathbf{D} = \begin{pmatrix} d_{11} & d_{12} & \cdots & d_{1t} \\ d_{21} & d_{22} & \cdots & d_{2t} \\ \vdots & \vdots & \ddots & \vdots \\ d_{t1} & d_{t2} & \cdots & d_{tt} \end{pmatrix}$$

从而解得不定乘数  $d_{11}, d_{22}, \dots, d_{tt}$  的值，根据方程组 6 可以得到相应估计量的标准差。

## 五、程序编写

### (一) 软件环境

使用 MWorks 进行程序设计，环境语言为 Julia。

### (二) 处理功能

读取 clv 表格中的数据列导入到 MWorks 中进行数据处理。

能够完整的进行等精度的九步骤求解，求出对应数据列的均值、标准差、残余误差、残余误差均值、残余误差标准差。通过残余误差校核法、残余误差观察法、不同计算标准差比较法三种方法来判断是否存在系统误差。通过  $3\sigma$  准则，罗曼诺夫斯基准则两种方法来剔除粗大误差，计算算术平均值的标准差和极限误差，并给出最终结果。

能够完整的进行不等精度的五步骤求解，求出对应加权算术平均值、残余误差、算术平均值的标准差和极限误差，并给出最终结果。

能够完整的进行矩阵法的最小二乘法估算，并给出估算值及其标准差。

### (三) 等精度数据处理

等精度数据处理按照上述理论写出代码，其代码如图 5-1所示

```
1  #/*****等精度数据处理代码*****/
2  # Copyright (c) 2024 J.S.E.,Yanshan University. All rights reserved.
3  # File Name: 等精度.jl
4  # Author: yinlong you
5  # Version: V1.0
6  # Date: 2024/5/23
7  # Description:
8  # 该文件用于实现等精度数据处理功能，处理来自不同传感器的数据。不同传感器应文件如下
9  # 1 -> 电压 (量纲:V, 最小到度0.001V)
10 # 2 -> 电流 (量纲:A, 最小到度0.001A)
11 # 3 -> 电压 (量纲:V, 最小到度0.001V)
12 # 4 -> 电压 (量纲:V, 最小到度0.001V)
13 *****/
14
15 using Printf
16 using Base
17 using CSV
18 using TSV
19 using TSV
20 using TSV
21 using TSV
22 using TSV
23 using TSV
24
25 #####打印数据#####
26 function print_array(array)
27     for i in 1:length(array)
28         print(round(array[i], digits=5))
29         if i % 10 == 0
30             print("\n")
31         end
32     end
33     print("\n")
34 end
35
36 #####计算小量误差#####
37 function calculate_number(number)
38     A = 1.0
39     for i in 1:5
40         number = 10*number
41         A = A / 10
42     end
43     return A
44 end
45
46 return 5
47
48 #####打印误差#####
49 function check_error(data_array, error_array, stand_number)
50     b = false
51     for i in 1:length(data_array)
52         if abs(error_array[i]) > stand_number
53             print("电压", A, "个", data_array[i], "含有粗大误差，请剔除")
54             b = true
55             return b
56         end
57     end
58     if b == false
59         print("无粗大误差")
60     end
61     return -1
62 end
63
64 #####计算误差#####
65 function find_t(number)
66     error = 0
67     for i in 1:10
68         fun(x) = exp(-(x^2)/2)
69         Q = integral(fun, 0, t)
70         Q = Q / sqrt(2*pi)
71         error = abs(Q - number)
72     end
73     return error
74 end
75
76 #####计算误差#####
77 function deal_data(data_array)
78     data_size = length(data_array)
79     while true
80         #第一步：求平均值
81         Data_mean = sum(data_array) / data_size
82         print("电压", A, "个", Data_mean, "的算术平均值为", Data_mean, "\n")
83         #第二步：求标准差
84         Error_array = zeros(size(data_array))
85         for i in 1:data_size
86             Error_array[i] = (data_array[i] - Data_mean)^2
87             Error_array[i] = sqrt(Error_array[i])
88             print("电压", A, "个", Error_array[i], "的残余误差为", Error_array[i], "\n")
89             #第三步：求残余误差
90             Error_array[i] = Error_array[i] - Data_mean
91             print("电压", A, "个", Error_array[i], "的残余误差为", Error_array[i], "\n")
92             #第四步：求残余误差均值
93             Error_mean = sum(Error_array) / data_size
94             print("电压", A, "个", Error_mean, "的残余误差均值为", Error_mean, "\n")
95             #第五步：求残余误差标准差
96             Error_std = sqrt(sum(Error_array^2) / (data_size - 1))
97             print("电压", A, "个", Error_std, "的残余误差标准差为", Error_std, "\n")
98             #第六步：求算术平均值的标准差
99             Data_std = Error_std / sqrt(data_size)
100             print("电压", A, "个", Data_std, "的算术平均值的标准差为", Data_std, "\n")
101             #第七步：求极限误差
102             Data_limit = 3 * Data_std
103             print("电压", A, "个", Data_limit, "的极限误差为", Data_limit, "\n")
104             #第八步：判断是否存在系统误差
105             #罗曼诺夫斯基准则
106             R = Data_limit / Data_std
107             R_table = [0.795, 0.854, 0.904, 0.951, 0.989, 1.028, 1.067, 1.106, 1.145, 1.183, 1.221, 1.259, 1.296, 1.333, 1.370, 1.407, 1.444, 1.481, 1.518, 1.555, 1.592, 1.629, 1.666, 1.703, 1.740, 1.777, 1.814, 1.851, 1.888, 1.925, 1.962, 1.999, 2.036, 2.073, 2.110, 2.147, 2.184, 2.221, 2.258, 2.295, 2.332, 2.369, 2.406, 2.443, 2.480, 2.517, 2.554, 2.591, 2.628, 2.665, 2.702, 2.739, 2.776, 2.813, 2.850, 2.887, 2.924, 2.961, 2.998, 3.035, 3.072, 3.109, 3.146, 3.183, 3.220, 3.257, 3.294, 3.331, 3.368, 3.405, 3.442, 3.479, 3.516, 3.553, 3.590, 3.627, 3.664, 3.701, 3.738, 3.775, 3.812, 3.849, 3.886, 3.923, 3.960, 3.997, 4.034, 4.071, 4.108, 4.145, 4.182, 4.219, 4.256, 4.293, 4.330, 4.367, 4.404, 4.441, 4.478, 4.515, 4.552, 4.589, 4.626, 4.663, 4.700, 4.737, 4.774, 4.811, 4.848, 4.885, 4.922, 4.959, 4.996, 5.033, 5.070, 5.107, 5.144, 5.181, 5.218, 5.255, 5.292, 5.329, 5.366, 5.403, 5.440, 5.477, 5.514, 5.551, 5.588, 5.625, 5.662, 5.699, 5.736, 5.773, 5.810, 5.847, 5.884, 5.921, 5.958, 5.995, 6.032, 6.069, 6.106, 6.143, 6.180, 6.217, 6.254, 6.291, 6.328, 6.365, 6.402, 6.439, 6.476, 6.513, 6.550, 6.587, 6.624, 6.661, 6.698, 6.735, 6.772, 6.809, 6.846, 6.883, 6.920, 6.957, 6.994, 7.031, 7.068, 7.105, 7.142, 7.179, 7.216, 7.253, 7.290, 7.327, 7.364, 7.401, 7.438, 7.475, 7.512, 7.549, 7.586, 7.623, 7.660, 7.697, 7.734, 7.771, 7.808, 7.845, 7.882, 7.919, 7.956, 7.993, 8.030, 8.067, 8.104, 8.141, 8.178, 8.215, 8.252, 8.289, 8.326, 8.363, 8.400, 8.437, 8.474, 8.511, 8.548, 8.585, 8.622, 8.659, 8.696, 8.733, 8.770, 8.807, 8.844, 8.881, 8.918, 8.955, 8.992, 9.029, 9.066, 9.103, 9.140, 9.177, 9.214, 9.251, 9.288, 9.325, 9.362, 9.399, 9.436, 9.473, 9.510, 9.547, 9.584, 9.621, 9.658, 9.695, 9.732, 9.769, 9.806, 9.843, 9.880, 9.917, 9.954, 9.991, 10.028, 10.065, 10.102, 10.139, 10.176, 10.213, 10.250, 10.287, 10.324, 10.361, 10.398, 10.435, 10.472, 10.509, 10.546, 10.583, 10.620, 10.657, 10.694, 10.731, 10.768, 10.805, 10.842, 10.879, 10.916, 10.953, 10.990, 11.027, 11.064, 11.101, 11.138, 11.175, 11.212, 11.249, 11.286, 11.323, 11.360, 11.397, 11.434, 11.471, 11.508, 11.545, 11.582, 11.619, 11.656, 11.693, 11.730, 11.767, 11.804, 11.841, 11.878, 11.915, 11.952, 11.989, 12.026, 12.063, 12.100, 12.137, 12.174, 12.211, 12.248, 12.285, 12.322, 12.359, 12.396, 12.433, 12.470, 12.507, 12.544, 12.581, 12.618, 12.655, 12.692, 12.729, 12.766, 12.803, 12.840, 12.877, 12.914, 12.951, 12.988, 13.025, 13.062, 13.099, 13.136, 13.173, 13.210, 13.247, 13.284, 13.321, 13.358, 13.395, 13.432, 13.469, 13.506, 13.543, 13.580, 13.617, 13.654, 13.691, 13.728, 13.765, 13.802, 13.839, 13.876, 13.913, 13.950, 13.987, 14.024, 14.061, 14.098, 14.135, 14.172, 14.209, 14.246, 14.283, 14.320, 14.357, 14.394, 14.431, 14.468, 14.505, 14.542, 14.579, 14.616, 14.653, 14.690, 14.727, 14.764, 14.801, 14.838, 14.875, 14.912, 14.949, 14.986, 15.023, 15.060, 15.097, 15.134, 15.171, 15.208, 15.245, 15.282, 15.319, 15.356, 15.393, 15.430, 15.467, 15.504, 15.541, 15.578, 15.615, 15.652, 15.689, 15.726, 15.763, 15.800, 15.837, 15.874, 15.911, 15.948, 15.985, 16.022, 16.059, 16.096, 16.133, 16.170, 16.207, 16.244, 16.281, 16.318, 16.355, 16.392, 16.429, 16.466, 16.503, 16.540, 16.577, 16.614, 16.651, 16.688, 16.725, 16.762, 16.799, 16.836, 16.873, 16.910, 16.947, 16.984, 17.021, 17.058, 17.095, 17.132, 17.169, 17.206, 17.243, 17.280, 17.317, 17.354, 17.391, 17.428, 17.465, 17.502, 17.539, 17.576, 17.613, 17.650, 17.687, 17.724, 17.761, 17.798, 17.835, 17.872, 17.909, 17.946, 17.983, 18.020, 18.057, 18.094, 18.131, 18.168, 18.205, 18.242, 18.279, 18.316, 18.353, 18.390, 18.427, 18.464, 18.501, 18.538, 18.575, 18.612, 18.649, 18.686, 18.723, 18.760, 18.797, 18.834, 18.871, 18.908, 18.945, 18.982, 19.019, 19.056, 19.093, 19.130, 19.167, 19.204, 19.241, 19.278, 19.315, 19.352, 19.389, 19.426, 19.463, 19.500, 19.537, 19.574, 19.611, 19.648, 19.685, 19.722, 19.759, 19.796, 19.833, 19.870, 19.907, 19.944, 19.981, 20.018, 20.055, 20.092, 20.129, 20.166, 20.203, 20.240, 20.277, 20.314, 20.351, 20.388, 20.425, 20.462, 20.499, 20.536, 20.573, 20.610, 20.647, 20.684, 20.721, 20.758, 20.795, 20.832, 20.869, 20.906, 20.943, 20.980, 21.017, 21.054, 21.091, 21.128, 21.165, 21.202, 21.239, 21.276, 21.313, 21.350, 21.387, 21.424, 21.461, 21.498, 21.535, 21.572, 21.609, 21.646, 21.683, 21.720, 21.757, 21.794, 21.831, 21.868, 21.905, 21.942, 21.979, 22.016, 22.053, 22.090, 22.127, 22.164, 22.201, 22.238, 22.275, 22.312, 22.349, 22.386, 22.423, 22.460, 22.497, 22.534, 22.571, 22.608, 22.645, 22.682, 22.719, 22.756, 22.793, 22.830, 22.867, 22.904, 22.941, 22.978, 23.015, 23.052, 23.089, 23.126, 23.163, 23.200, 23.237, 23.274, 23.311, 23.348, 23.385, 23.422, 23.459, 23.496, 23.533, 23.570, 23.607, 23.644, 23.681, 23.718, 23.755, 23.792, 23.829, 23.866, 23.903, 23.940, 23.977, 24.014, 24.051, 24.088, 24.125, 24.162, 24.199, 24.236, 24.273, 24.310, 24.347, 24.384, 24.421, 24.458, 24.495, 24.532, 24.569, 24.606, 24.643, 24.680, 24.717, 24.754, 24.791, 24.828, 24.865, 24.902, 24.939, 24.976, 25.013, 25.050, 25.087, 25.124, 25.161, 25.198, 25.235, 25.272, 25.309, 25.346, 25.383, 25.420, 25.457, 25.494, 25.531, 25.568, 25.605, 25.642, 25.679, 25.716, 25.753, 25.790, 25.827, 25.864, 25.901, 25.938, 25.975, 26.012, 26.049, 26.086, 26.123, 26.160, 26.197, 26.234, 26.271, 26.308, 26.345, 26.382, 26.419, 26.456, 26.493, 26.530, 26.567, 26.604, 26.641, 26.678, 26.715, 26.752, 26.789, 26.826, 26.863, 26.900, 26.937, 26.974, 27.011, 27.048, 27.085, 27.122, 27.159, 27.196, 27.233, 27.270, 27.307, 27.344, 27.381, 27.418, 27.455, 27.492, 27.529, 27.566, 27.603, 27.640, 27.677, 27.714, 27.751, 27.788, 27.825, 27.862, 27.899, 27.936, 27.973, 28.010, 28.047, 28.084, 28.121, 28.158, 28.195, 28.232, 28.269, 28.306, 28.343, 28.380, 28.417, 28.454, 28.491, 28.528, 28.565, 28.602, 28.639, 28.676, 28.713, 28.750, 28.787, 28.824, 28.861, 28.898, 28.935, 28.972, 29.009, 29.046, 29.083, 29.120, 29.157, 29.194, 29.231, 29.268, 29.305, 29.342, 29.379, 29.416, 29.453, 29.490, 29.527, 29.564, 29.601, 29.638, 29.675, 29.712, 29.749, 29.786, 29.823, 29.860, 29.897, 29.934, 29.971, 30.008, 30.045, 30.082, 30.119, 30.156, 30.193, 30.230, 30.267, 30.304, 30.341, 30.378, 30.415, 30.452, 30.489, 30.526, 30.563, 30.600, 30.637, 30.674, 30.711, 30.748, 30.785, 30.822, 30.859, 30.896, 30.933, 30.970, 31.007, 31.044, 31.081, 31.118, 31.155, 31.192, 31.229, 31.266, 31.303, 31.340, 31.377, 31.414, 31.451, 31.488, 31.525, 31.562, 31.599, 31.636, 31.673, 31.710, 31.747, 31.784, 31.821, 31.858, 31.895, 31.932, 31.969, 32.006, 32.043, 32.080, 32.117, 32.154, 32.191, 32.228, 32.265, 32.302, 32.339, 32.376, 32.413, 32.450, 32.487, 32.524, 32.561, 32.598, 32.635, 32.672, 32.709, 32.746, 32.783, 32.820, 32.857, 32.894, 32.931, 32.968, 33.005, 33.042, 33.079, 33.116, 33.153, 33.190, 33.227, 33.264, 33.301, 33.338, 33.375, 33.412, 33.449, 33.486, 33.523, 33.560, 33.597, 33.634, 33.671, 33.708, 33.745, 33.782, 33.819, 33.856, 33.893, 33.930, 33.967, 34.004, 34.041, 34.078, 34.115, 34.152, 34.189, 34.226, 34.263, 34.300, 34.337, 34.374, 34.411, 34.448, 34.485, 34.522, 34.559, 34.596, 34.633, 34.670, 34.707, 34.744, 34.781, 34.818, 34.855, 34.892, 34.929, 34.966, 35.003, 35.040, 35.077, 35.114, 35.151, 35.188, 35.225, 35.262, 35.299, 35.336, 35.373, 35.410, 35.447, 35.484, 35.521, 35.558, 35.595, 35.632, 35.669, 35.706, 35.743, 35.780, 35.817, 35.854, 35.891, 35.928, 35.965, 36.002, 36.039, 36.076, 36.113, 36.150, 36.187, 36.224, 36.261, 36.298, 36.335, 36.372, 36.409, 36.446, 36.483, 36.520, 36.557, 36.594, 36.631, 36.668, 36.705, 36.742, 36.779, 36.816, 36.853, 36.890, 36.927, 36.964, 37.001, 37.038, 37.075, 37.112, 37.149, 37.186, 37.223, 37.260, 37.297, 37.334, 37.371, 37.408, 37.445, 37.482, 37.519, 37.556, 37.593, 37.630, 37.667, 37.704, 37.741, 37.778, 37.815, 37.852, 37.889, 37.926, 37.963, 38.000, 38.037, 38.074, 38.111, 38.148, 38.185, 38.222, 38.259, 38.296, 38.333, 38.370, 38.407, 38.444, 38.481, 38.518, 38.555, 38.592, 38.629, 38.666, 38.703, 38.740, 38.777, 38.814, 38.851, 38.888, 38.925, 38.962, 39.000, 39.036, 39.073, 39.110, 39.147, 39.184, 39.221, 39.258, 39.295, 39.332, 39.369, 39.406, 39.443, 39.480, 39.517, 39.554, 39.591, 39.628, 39.665, 39.702, 39.739, 39.776, 39.813, 39.850, 39.887, 39.924, 39.961, 40.000, 40.036, 40.073, 40.110, 40.147, 40.184, 40.221, 40.258, 40.295, 40.332, 40.369, 40.406, 40.443, 40.480, 40.517, 40.554, 40.591, 40.628, 40.665, 40.702, 40.739, 40.776, 40.813, 40.850, 40.887, 40.924, 40.961, 41.000, 41.036, 41.073, 41.110, 41.147, 41.184, 41.221, 41.258, 41.295, 41.332, 41.369, 41.406, 41.443, 41.480, 41.517, 41.554, 41.591, 41.628, 41.665, 41.702, 41.739, 41.776, 41.813, 41.850, 41.887, 41.924, 41.961, 42.000, 42.036, 42.073, 42.110, 42.147, 42.184, 42.221, 42.258, 42.295, 42.332, 42.369, 42.406, 42.443, 42.480, 42.517, 42.554, 42.591, 42.628, 42.665, 42.702, 42.739, 42.776, 42.813, 42.850, 42.887, 42.924, 42.961, 43.000, 43.036, 43.073, 43.110, 43.147, 43.184, 43.221, 43.258, 43.295, 43.332, 43.369, 43.406, 43.443, 43.480, 43.517, 43.554, 43.591, 43.628, 43.665, 43.702, 43.739, 43.776, 43.813, 43.850, 43.887, 43.924, 43.961, 44.000, 44.036, 44.073, 44.110, 44.147, 44.184, 44.221, 44.258, 44.295, 44.332, 44.369, 44.406, 44.443, 44.480, 44.517, 44.554, 44.591, 44.628, 44.665, 44.702, 44.739, 44.776, 44.813, 44.850, 44.887, 44.924, 44.961, 45.000, 45.036, 45.073, 45.110, 45.147, 45.184, 45.221, 45.258, 45.295, 45.332, 45.369, 45.406, 45.443, 45.480, 45.517, 45.554, 45.591, 45.628, 45.665, 45.702, 45.739, 45.776, 45.813, 45.850, 45.887, 45.924, 45.961, 46.000, 46.036, 46.073, 46.110, 46.147, 46.184, 46.221, 46.258, 46.295, 46.332, 46.369, 46.406, 46.443, 46.480, 46.517, 46.554, 46.591, 46.628, 46.665, 46.702, 46.739, 46.776, 46.813, 46.850, 46.887, 46.924, 46.961, 47.000, 47.036, 47.073, 47.110, 47.147, 47.184, 47.221, 47.258, 47.295, 47.332, 47.369, 47.406, 47.443, 47.480, 47.517, 47.554, 47.591, 47.628, 47.665, 47.702, 47.739, 47.776, 47.813, 47.850, 47.887, 47.924, 47.961, 48.000, 48.036, 48.073, 48.110, 48.147, 48.184, 48.221, 48.258, 48.295, 48.332, 48.369, 48.406, 48.443, 48.480, 48.517, 48.554, 48.591, 48.628, 48.665, 48.702, 48.739, 48.776, 48.813, 48.850, 48.887, 48.924, 48.961, 49.000, 49.036, 49.073, 49.110, 49.147, 49.184, 49.221, 49.258, 49.295, 49.332, 49.369, 49.406, 49.443, 49.480, 49.517, 49.554, 49.591, 49.628, 49.665, 49.702, 49.739, 49.776, 49.813, 49.850, 49.887, 49.924, 49.961, 50.000, 50.036, 50.073, 50.110, 50.147, 50.184, 50.221, 50.258, 50.295, 50.332, 50.369, 50.406, 50.443, 50.480, 50.517, 50.554, 50.591, 50.628, 50.665, 50.702, 50.739, 50.776, 50.813, 50.850, 50.887, 50.924, 5
```

代码流程图如下：

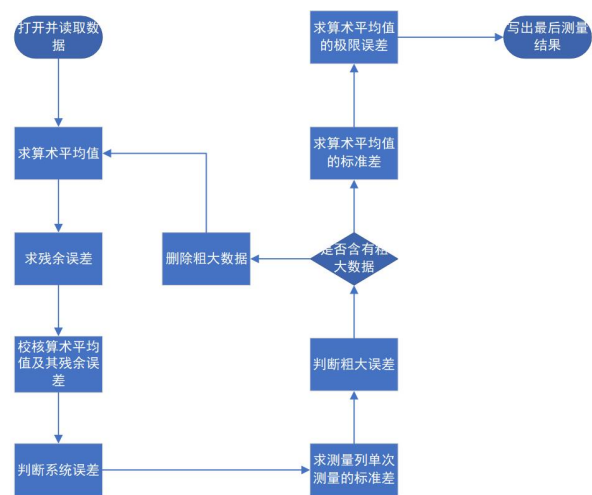


图 5-2: 等精度数据处理流程图

值得注意的是：

1. 代码可以自动读取数据，并直接根据读取的数据生成对应九个步骤的数据处理操作结果。
2. 第四步，判断系统误差时，使用了残余误差校核法和不同公式计算标准差法两种方法进行系统误差判断，能更准确的确认系统误差的存在与否。
3. 第六步，判别粗大误差，使用了罗曼诺夫斯基准则、 $3\sigma$  准则和格罗布斯准则三种方法进行系统误差判断，能更准确的确认系统误差的存在与否。
4. 当在第六步确认出现系统误差后，代码可以自行剔除粗大误差数据，并回到第一步重新进行等精度测量。剔除行为直到不存在粗大误差或是数据少于两个时停止。
5. 第八步，求算术平均值的极限误差时，会根据测量量的个数选择合适、合理的方式进行
6. 当使用正态分布计算算术平均值的极限误差时， $t$  的取值由积分函数反解求得，因此相比于查表法更加精确。

#### (四) 不等精度数据处理

不等精度数据处理按照上述理论写出代码，其代码如图 5-3所示

```

1  # Copyright (C) 2004 I.S.E., Yanshen University. All rights reserved.
2  # File name: 不等精度数据
3  # Author: YSlong you
4  # Version: V1.0
5  # Date: 2004-12-21
6  # Description
7  # 根据不等精度测量数据求平均值 (为了减小误差, 不采用加权平均, 而是采用等精度测量)
8  # 求平均值并输出结果
9  # 求平均值并输出结果
10 # 求平均值并输出结果
11 using System;
12 using System.Collections.Generic;
13 using System.Linq;
14 using System.Text;
15 using System.Threading.Tasks;
16 using System.Windows.Forms;
17 using System.Drawing;
18 using System.Data;
19 using System.Data.SqlClient;
20
21 #region 定义数据库连接字符串
22 string connStr = "Server=.;Database=.;User=sa;Password=.;";
23 #endregion
24
25 #region 数据库操作
26 public static void ExecuteNonQuery(string sql)
27 {
28     using (SqlConnection conn = new SqlConnection(connStr))
29     {
30         conn.Open();
31         using (SqlCommand cmd = new SqlCommand(sql, conn))
32         {
33             cmd.ExecuteNonQuery();
34         }
35     }
36 }
37
38 public static void ExecuteQuery(string sql, out string result)
39 {
40     using (SqlConnection conn = new SqlConnection(connStr))
41     {
42         conn.Open();
43         using (SqlCommand cmd = new SqlCommand(sql, conn))
44         {
45             SqlDataReader reader = cmd.ExecuteReader();
46             result = "";
47             while (reader.Read())
48             {
49                 result += reader.GetString(0) + "\n";
50             }
51         }
52     }
53 }
54
55 public static void ExecuteQuery(string sql, out double result)
56 {
57     using (SqlConnection conn = new SqlConnection(connStr))
58     {
59         conn.Open();
60         using (SqlCommand cmd = new SqlCommand(sql, conn))
61         {
62             SqlDataReader reader = cmd.ExecuteReader();
63             result = 0;
64             while (reader.Read())
65             {
66                 result += reader.GetDouble(0);
67             }
68         }
69     }
70 }
71
72 public static void ExecuteQuery(string sql, out int result)
73 {
74     using (SqlConnection conn = new SqlConnection(connStr))
75     {
76         conn.Open();
77         using (SqlCommand cmd = new SqlCommand(sql, conn))
78         {
79             SqlDataReader reader = cmd.ExecuteReader();
80             result = 0;
81             while (reader.Read())
82             {
83                 result += reader.GetInt32(0);
84             }
85         }
86     }
87 }
88
89 public static void ExecuteQuery(string sql, out decimal result)
90 {
91     using (SqlConnection conn = new SqlConnection(connStr))
92     {
93         conn.Open();
94         using (SqlCommand cmd = new SqlCommand(sql, conn))
95         {
96             SqlDataReader reader = cmd.ExecuteReader();
97             result = 0;
98             while (reader.Read())
99             {
100                 result += reader.GetDecimal(0);
101             }
102         }
103     }
104 }
105
106 public static void ExecuteQuery(string sql, out string result)
107 {
108     using (SqlConnection conn = new SqlConnection(connStr))
109     {
110         conn.Open();
111         using (SqlCommand cmd = new SqlCommand(sql, conn))
112         {
113             SqlDataReader reader = cmd.ExecuteReader();
114             result = "";
115             while (reader.Read())
116             {
117                 result += reader.GetString(0) + "\n";
118             }
119         }
120     }
121 }
122
123 public static void ExecuteQuery(string sql, out double result)
124 {
125     using (SqlConnection conn = new SqlConnection(connStr))
126     {
127         conn.Open();
128         using (SqlCommand cmd = new SqlCommand(sql, conn))
129         {
130             SqlDataReader reader = cmd.ExecuteReader();
131             result = 0;
132             while (reader.Read())
133             {
134                 result += reader.GetDouble(0);
135             }
136         }
137     }
138 }
139
140 public static void ExecuteQuery(string sql, out int result)
141 {
142     using (SqlConnection conn = new SqlConnection(connStr))
143     {
144         conn.Open();
145         using (SqlCommand cmd = new SqlCommand(sql, conn))
146         {
147             SqlDataReader reader = cmd.ExecuteReader();
148             result = 0;
149             while (reader.Read())
150             {
151                 result += reader.GetInt32(0);
152             }
153         }
154     }
155 }
156
157 public static void ExecuteQuery(string sql, out decimal result)
158 {
159     using (SqlConnection conn = new SqlConnection(connStr))
160     {
161         conn.Open();
162         using (SqlCommand cmd = new SqlCommand(sql, conn))
163         {
164             SqlDataReader reader = cmd.ExecuteReader();
165             result = 0;
166             while (reader.Read())
167             {
168                 result += reader.GetDecimal(0);
169             }
170         }
171     }
172 }
173
174 public static void ExecuteQuery(string sql, out string result)
175 {
176     using (SqlConnection conn = new SqlConnection(connStr))
177     {
178         conn.Open();
179         using (SqlCommand cmd = new SqlCommand(sql, conn))
180         {
181             SqlDataReader reader = cmd.ExecuteReader();
182             result = "";
183             while (reader.Read())
184             {
185                 result += reader.GetString(0) + "\n";
186             }
187         }
188     }
189 }
190
191 public static void ExecuteQuery(string sql, out double result)
192 {
193     using (SqlConnection conn = new SqlConnection(connStr))
194     {
195         conn.Open();
196         using (SqlCommand cmd = new SqlCommand(sql, conn))
197         {
198             SqlDataReader reader = cmd.ExecuteReader();
199             result = 0;
200             while (reader.Read())
201             {
202                 result += reader.GetDouble(0);
203             }
204         }
205     }
206 }
207
208 public static void ExecuteQuery(string sql, out int result)
209 {
210     using (SqlConnection conn = new SqlConnection(connStr))
211     {
212         conn.Open();
213         using (SqlCommand cmd = new SqlCommand(sql, conn))
214         {
215             SqlDataReader reader = cmd.ExecuteReader();
216             result = 0;
217             while (reader.Read())
218             {
219                 result += reader.GetInt32(0);
220             }
221         }
222     }
223 }
224
225 public static void ExecuteQuery(string sql, out decimal result)
226 {
227     using (SqlConnection conn = new SqlConnection(connStr))
228     {
229         conn.Open();
230         using (SqlCommand cmd = new SqlCommand(sql, conn))
231         {
232             SqlDataReader reader = cmd.ExecuteReader();
233             result = 0;
234             while (reader.Read())
235             {
236                 result += reader.GetDecimal(0);
237             }
238         }
239     }
240 }
241
242 public static void ExecuteQuery(string sql, out string result)
243 {
244     using (SqlConnection conn = new SqlConnection(connStr))
245     {
246         conn.Open();
247         using (SqlCommand cmd = new SqlCommand(sql, conn))
248         {
249             SqlDataReader reader = cmd.ExecuteReader();
250             result = "";
251             while (reader.Read())
252             {
253                 result += reader.GetString(0) + "\n";
254             }
255         }
256     }
257 }
258
259 public static void ExecuteQuery(string sql, out double result)
260 {
261     using (SqlConnection conn = new SqlConnection(connStr))
262     {
263         conn.Open();
264         using (SqlCommand cmd = new SqlCommand(sql, conn))
265         {
266             SqlDataReader reader = cmd.ExecuteReader();
267             result = 0;
268             while (reader.Read())
269             {
270                 result += reader.GetDouble(0);
271             }
272         }
273     }
274 }
275
276 public static void ExecuteQuery(string sql, out int result)
277 {
278     using (SqlConnection conn = new SqlConnection(connStr))
279     {
280         conn.Open();
281         using (SqlCommand cmd = new SqlCommand(sql, conn))
282         {
283             SqlDataReader reader = cmd.ExecuteReader();
284             result = 0;
285             while (reader.Read())
286             {
287                 result += reader.GetInt32(0);
288             }
289         }
290     }
291 }
292
293 public static void ExecuteQuery(string sql, out decimal result)
294 {
295     using (SqlConnection conn = new SqlConnection(connStr))
296     {
297         conn.Open();
298         using (SqlCommand cmd = new SqlCommand(sql, conn))
299         {
300             SqlDataReader reader = cmd.ExecuteReader();
301             result = 0;
302             while (reader.Read())
303             {
304                 result += reader.GetDecimal(0);
305             }
306         }
307     }
308 }
309
310 public static void ExecuteQuery(string sql, out string result)
311 {
312     using (SqlConnection conn = new SqlConnection(connStr))
313     {
314         conn.Open();
315         using (SqlCommand cmd = new SqlCommand(sql, conn))
316         {
317             SqlDataReader reader = cmd.ExecuteReader();
318             result = "";
319             while (reader.Read())
320             {
321                 result += reader.GetString(0) + "\n";
322             }
323         }
324     }
325 }
326
327 public static void ExecuteQuery(string sql, out double result)
328 {
329     using (SqlConnection conn = new SqlConnection(connStr))
330     {
331         conn.Open();
332         using (SqlCommand cmd = new SqlCommand(sql, conn))
333         {
334             SqlDataReader reader = cmd.ExecuteReader();
335             result = 0;
336             while (reader.Read())
337             {
338                 result += reader.GetDouble(0);
339             }
340         }
341     }
342 }
343
344 public static void ExecuteQuery(string sql, out int result)
345 {
346     using (SqlConnection conn = new SqlConnection(connStr))
347     {
348         conn.Open();
349         using (SqlCommand cmd = new SqlCommand(sql, conn))
350         {
351             SqlDataReader reader = cmd.ExecuteReader();
352             result = 0;
353             while (reader.Read())
354             {
355                 result += reader.GetInt32(0);
356             }
357         }
358     }
359 }
360
361 public static void ExecuteQuery(string sql, out decimal result)
362 {
363     using (SqlConnection conn = new SqlConnection(connStr))
364     {
365         conn.Open();
366         using (SqlCommand cmd = new SqlCommand(sql, conn))
367         {
368             SqlDataReader reader = cmd.ExecuteReader();
369             result = 0;
370             while (reader.Read())
371             {
372                 result += reader.GetDecimal(0);
373             }
374         }
375     }
376 }
377
378 public static void ExecuteQuery(string sql, out string result)
379 {
380     using (SqlConnection conn = new SqlConnection(connStr))
381     {
382         conn.Open();
383         using (SqlCommand cmd = new SqlCommand(sql, conn))
384         {
385             SqlDataReader reader = cmd.ExecuteReader();
386             result = "";
387             while (reader.Read())
388             {
389                 result += reader.GetString(0) + "\n";
390             }
391         }
392     }
393 }
394
395 public static void ExecuteQuery(string sql, out double result)
396 {
397     using (SqlConnection conn = new SqlConnection(connStr))
398     {
399         conn.Open();
400         using (SqlCommand cmd = new SqlCommand(sql, conn))
401         {
402             SqlDataReader reader = cmd.ExecuteReader();
403             result = 0;
404             while (reader.Read())
405             {
406                 result += reader.GetDouble(0);
407             }
408         }
409     }
410 }
411
412 public static void ExecuteQuery(string sql, out int result)
413 {
414     using (SqlConnection conn = new SqlConnection(connStr))
415     {
416         conn.Open();
417         using (SqlCommand cmd = new SqlCommand(sql, conn))
418         {
419             SqlDataReader reader = cmd.ExecuteReader();
420             result = 0;
421             while (reader.Read())
422             {
423                 result += reader.GetInt32(0);
424             }
425         }
426     }
427 }
428
429 public static void ExecuteQuery(string sql, out decimal result)
430 {
431     using (SqlConnection conn = new SqlConnection(connStr))
432     {
433         conn.Open();
434         using (SqlCommand cmd = new SqlCommand(sql, conn))
435         {
436             SqlDataReader reader = cmd.ExecuteReader();
437             result = 0;
438             while (reader.Read())
439             {
440                 result += reader.GetDecimal(0);
441             }
442         }
443     }
444 }
445
446 public static void ExecuteQuery(string sql, out string result)
447 {
448     using (SqlConnection conn = new SqlConnection(connStr))
449     {
450         conn.Open();
451         using (SqlCommand cmd = new SqlCommand(sql, conn))
452         {
453             SqlDataReader reader = cmd.ExecuteReader();
454             result = "";
455             while (reader.Read())
456             {
457                 result += reader.GetString(0) + "\n";
458             }
459         }
460     }
461 }
462
463 public static void ExecuteQuery(string sql, out double result)
464 {
465     using (SqlConnection conn = new SqlConnection(connStr))
466     {
467         conn.Open();
468         using (SqlCommand cmd = new SqlCommand(sql, conn))
469         {
470             SqlDataReader reader = cmd.ExecuteReader();
471             result = 0;
472             while (reader.Read())
473             {
474                 result += reader.GetDouble(0);
475             }
476         }
477     }
478 }
479
480 public static void ExecuteQuery(string sql, out int result)
481 {
482     using (SqlConnection conn = new SqlConnection(connStr))
483     {
484         conn.Open();
485         using (SqlCommand cmd = new SqlCommand(sql, conn))
486         {
487             SqlDataReader reader = cmd.ExecuteReader();
488             result = 0;
489             while (reader.Read())
490             {
491                 result += reader.GetInt32(0);
492             }
493         }
494     }
495 }
496
497 public static void ExecuteQuery(string sql, out decimal result)
498 {
499     using (SqlConnection conn = new SqlConnection(connStr))
500     {
501         conn.Open();
502         using (SqlCommand cmd = new SqlCommand(sql, conn))
503         {
504             SqlDataReader reader = cmd.ExecuteReader();
505             result = 0;
506             while (reader.Read())
507             {
508                 result += reader.GetDecimal(0);
509             }
510         }
511     }
512 }
513
514 public static void ExecuteQuery(string sql, out string result)
515 {
516     using (SqlConnection conn = new SqlConnection(connStr))
517     {
518         conn.Open();
519         using (SqlCommand cmd = new SqlCommand(sql, conn))
520         {
521             SqlDataReader reader = cmd.ExecuteReader();
522             result = "";
523             while (reader.Read())
524             {
525                 result += reader.GetString(0) + "\n";
526             }
527         }
528     }
529 }
530
531 public static void ExecuteQuery(string sql, out double result)
532 {
533     using (SqlConnection conn = new SqlConnection(connStr))
534     {
535         conn.Open();
536         using (SqlCommand cmd = new SqlCommand(sql, conn))
537         {
538             SqlDataReader reader = cmd.ExecuteReader();
539             result = 0;
540             while (reader.Read())
541             {
542                 result += reader.GetDouble(0);
543             }
544         }
545     }
546 }
547
548 public static void ExecuteQuery(string sql, out int result)
549 {
550     using (SqlConnection conn = new SqlConnection(connStr))
551     {
552         conn.Open();
553         using (SqlCommand cmd = new SqlCommand(sql, conn))
554         {
555             SqlDataReader reader = cmd.ExecuteReader();
556             result = 0;
557             while (reader.Read())
558             {
559                 result += reader.GetInt32(0);
560             }
561         }
562     }
563 }
564
565 public static void ExecuteQuery(string sql, out decimal result)
566 {
567     using (SqlConnection conn = new SqlConnection(connStr))
568     {
569         conn.Open();
570         using (SqlCommand cmd = new SqlCommand(sql, conn))
571         {
572             SqlDataReader reader = cmd.ExecuteReader();
573             result = 0;
574             while (reader.Read())
575             {
576                 result += reader.GetDecimal(0);
577             }
578         }
579     }
580 }
581
582 public static void ExecuteQuery(string sql, out string result)
583 {
584     using (SqlConnection conn = new SqlConnection(connStr))
585     {
586         conn.Open();
587         using (SqlCommand cmd = new SqlCommand(sql, conn))
588         {
589             SqlDataReader reader = cmd.ExecuteReader();
590             result = "";
591             while (reader.Read())
592             {
593                 result += reader.GetString(0) + "\n";
594             }
595         }
596     }
597 }
598
599 public static void ExecuteQuery(string sql, out double result)
600 {
601     using (SqlConnection conn = new SqlConnection(connStr))
602     {
603         conn.Open();
604         using (SqlCommand cmd = new SqlCommand(sql, conn))
605         {
606             SqlDataReader reader = cmd.ExecuteReader();
607             result = 0;
608             while (reader.Read())
609             {
610                 result += reader.GetDouble(0);
611             }
612         }
613     }
614 }
615
616 public static void ExecuteQuery(string sql, out int result)
617 {
618     using (SqlConnection conn = new SqlConnection(connStr))
619     {
620         conn.Open();
621         using (SqlCommand cmd = new SqlCommand(sql, conn))
622         {
623             SqlDataReader reader = cmd.ExecuteReader();
624             result = 0;
625             while (reader.Read())
626             {
627                 result += reader.GetInt32(0);
628             }
629         }
630     }
631 }
632
633 public static void ExecuteQuery(string sql, out decimal result)
634 {
635     using (SqlConnection conn = new SqlConnection(connStr))
636     {
637         conn.Open();
638         using (SqlCommand cmd = new SqlCommand(sql, conn))
639         {
640             SqlDataReader reader = cmd.ExecuteReader();
641             result = 0;
642             while (reader.Read())
643             {
644                 result += reader.GetDecimal(0);
645             }
646         }
647     }
648 }
649
650 public static void ExecuteQuery(string sql, out string result)
651 {
652     using (SqlConnection conn = new SqlConnection(connStr))
653     {
654         conn.Open();
655         using (SqlCommand cmd = new SqlCommand(sql, conn))
656         {
657             SqlDataReader reader = cmd.ExecuteReader();
658             result = "";
659             while (reader.Read())
660             {
661                 result += reader.GetString(0) + "\n";
662             }
663         }
664     }
665 }
666
667 public static void ExecuteQuery(string sql, out double result)
668 {
669     using (SqlConnection conn = new SqlConnection(connStr))
670     {
671         conn.Open();
672         using (SqlCommand cmd = new SqlCommand(sql, conn))
673         {
674             SqlDataReader reader = cmd.ExecuteReader();
675             result = 0;
676             while (reader.Read())
677             {
678                 result += reader.GetDouble(0);
679             }
680         }
681     }
682 }
683
684 public static void ExecuteQuery(string sql, out int result)
685 {
686     using (SqlConnection conn = new SqlConnection(connStr))
687     {
688         conn.Open();
689         using (SqlCommand cmd = new SqlCommand(sql, conn))
690         {
691             SqlDataReader reader = cmd.ExecuteReader();
692             result = 0;
693             while (reader.Read())
694             {
695                 result += reader.GetInt32(0);
696             }
697         }
698     }
699 }
700
701 public static void ExecuteQuery(string sql, out decimal result)
702 {
703     using (SqlConnection conn = new SqlConnection(connStr))
704     {
705         conn.Open();
706         using (SqlCommand cmd = new SqlCommand(sql, conn))
707         {
708             SqlDataReader reader = cmd.ExecuteReader();
709             result = 0;
710             while (reader.Read())
711             {
712                 result += reader.GetDecimal(0);
713             }
714         }
715     }
716 }
717
718 public static void ExecuteQuery(string sql, out string result)
719 {
720     using (SqlConnection conn = new SqlConnection(connStr))
721     {
722         conn.Open();
723         using (SqlCommand cmd = new SqlCommand(sql, conn))
724         {
725             SqlDataReader reader = cmd.ExecuteReader();
726             result = "";
727             while (reader.Read())
728             {
729                 result += reader.GetString(0) + "\n";
730             }
731         }
732     }
733 }
734
735 public static void ExecuteQuery(string sql, out double result)
736 {
737     using (SqlConnection conn = new SqlConnection(connStr))
738     {
739         conn.Open();
740         using (SqlCommand cmd = new SqlCommand(sql, conn))
741         {
742             SqlDataReader reader = cmd.ExecuteReader();
743             result = 0;
744             while (reader.Read())
745             {
746                 result += reader.GetDouble(0);
747             }
748         }
749     }
750 }
751
752 public static void ExecuteQuery(string sql, out int result)
753 {
754     using (SqlConnection conn = new SqlConnection(connStr))
755     {
756         conn.Open();
757         using (SqlCommand cmd = new SqlCommand(sql, conn))
758         {
759             SqlDataReader reader = cmd.ExecuteReader();
760             result = 0;
761             while (reader.Read())
762             {
763                 result += reader.GetInt32(0);
764             }
765         }
766     }
767 }
768
769 public static void ExecuteQuery(string sql, out decimal result)
770 {
771     using (SqlConnection conn = new SqlConnection(connStr))
772     {
773         conn.Open();
774         using (SqlCommand cmd = new SqlCommand(sql, conn))
775         {
776             SqlDataReader reader = cmd.ExecuteReader();
777             result = 0;
778             while (reader.Read())
779             {
780                 result += reader.GetDecimal(0);
781             }
782         }
783     }
784 }
785
786 public static void ExecuteQuery(string sql, out string result)
787 {
788     using (SqlConnection conn = new SqlConnection(connStr))
789     {
790         conn.Open();
791         using (SqlCommand cmd = new SqlCommand(sql, conn))
792         {
793             SqlDataReader reader = cmd.ExecuteReader();
794             result = "";
795             while (reader.Read())
796             {
797                 result += reader.GetString(0) + "\n";
798             }
799         }
800     }
801 }
802
803 public static void ExecuteQuery(string sql, out double result)
804 {
805     using (SqlConnection conn = new SqlConnection(connStr))
806     {
807         conn.Open();
808         using (SqlCommand cmd = new SqlCommand(sql, conn))
809         {
810             SqlDataReader reader = cmd.ExecuteReader();
811             result = 0;
812             while (reader.Read())
813             {
814                 result += reader.GetDouble(0);
815             }
816         }
817     }
818 }
819
820 public static void ExecuteQuery(string sql, out int result)
821 {
822     using (SqlConnection conn = new SqlConnection(connStr))
823     {
824         conn.Open();
825         using (SqlCommand cmd = new SqlCommand(sql, conn))
826         {
827             SqlDataReader reader = cmd.ExecuteReader();
828             result = 0;
829             while (reader.Read())
830             {
831                 result += reader.GetInt32(0);
832             }
833         }
834     }
835 }
836
837 public static void ExecuteQuery(string sql, out decimal result)
838 {
839     using (SqlConnection conn = new SqlConnection(connStr))
840     {
841         conn.Open();
842         using (SqlCommand cmd = new SqlCommand(sql, conn))
843         {
844             SqlDataReader reader = cmd.ExecuteReader();
845             result = 0;
846             while (reader.Read())
847             {
848                 result += reader.GetDecimal(0);
849             }
850         }
851     }
852 }
853
854 public static void ExecuteQuery(string sql, out string result)
855 {
856     using (SqlConnection conn = new SqlConnection(connStr))
857     {
858         conn.Open();
859         using (SqlCommand cmd = new SqlCommand(sql, conn))
860         {
861             SqlDataReader reader = cmd.ExecuteReader();
862             result = "";
863             while (reader.Read())
864             {
865                 result += reader.GetString(0) + "\n";
866             }
867         }
868     }
869 }
870
871 public static void ExecuteQuery(string sql, out double result)
872 {
873     using (SqlConnection conn = new SqlConnection(connStr))
874     {
875         conn.Open();
876         using (SqlCommand cmd = new SqlCommand(sql, conn))
877         {
878             SqlDataReader reader = cmd.ExecuteReader();
879             result = 0;
880             while (reader.Read())
881             {
882                 result += reader.GetDouble(0);
883             }
884         }
885     }
886 }
887
888 public static void ExecuteQuery(string sql, out int result)
889 {
890     using (SqlConnection conn = new SqlConnection(connStr))
891     {
892         conn.Open();
893         using (SqlCommand cmd = new SqlCommand(sql, conn))
894         {
895             SqlDataReader reader = cmd.ExecuteReader();
896             result = 0;
897             while (reader.Read())
898             {
899                 result += reader.GetInt32(0);
900             }
901         }
902     }
903 }
904
905 public static void ExecuteQuery(string sql, out decimal result)
906 {
907     using (SqlConnection conn = new SqlConnection(connStr))
908     {
909         conn.Open();
910         using (SqlCommand cmd = new SqlCommand(sql, conn))
911         {
912             SqlDataReader reader = cmd.ExecuteReader();
913             result = 0;
914             while (reader.Read())
915             {
916                 result += reader.GetDecimal(0);
917             }
918         }
919     }
920 }
921
922 public static void ExecuteQuery(string sql, out string result)
923 {
924     using (SqlConnection conn = new SqlConnection(connStr))
925     {
926         conn.Open();
927         using (SqlCommand cmd = new SqlCommand(sql, conn))
928         {
929             SqlDataReader reader = cmd.ExecuteReader();
930             result = "";
931             while (reader.Read())
932             {
933                 result += reader.GetString(0) + "\n";
934             }
935         }
936     }
937 }
938
939 public static void ExecuteQuery(string sql, out double result)
940 {
941     using (SqlConnection conn = new SqlConnection(connStr))
942     {
943         conn.Open();
944         using (SqlCommand cmd = new SqlCommand(sql, conn))
945         {
946             SqlDataReader reader = cmd.ExecuteReader();
947             result = 0;
948             while (reader.Read())
949             {
950                 result += reader.GetDouble(0);
951             }
952         }
953     }
954 }
955
956 public static void ExecuteQuery(string sql, out int result)
957 {
958     using (SqlConnection conn = new SqlConnection(connStr))
959     {
960         conn.Open();
961         using (SqlCommand cmd = new SqlCommand(sql, conn))
962         {
963             SqlDataReader reader = cmd.ExecuteReader();
964             result = 0;
965             while (reader.Read())
966             {
967                 result += reader.GetInt32(0);
968             }
969         }
970     }
971 }
972
973 public static void ExecuteQuery(string sql, out decimal result)
974 {
975     using (SqlConnection conn = new SqlConnection(connStr))
976     {
977         conn.Open();
978         using (SqlCommand cmd = new SqlCommand(sql, conn))
979         {
980             SqlDataReader reader = cmd.ExecuteReader();
981             result = 0;
982             while (reader.Read())
983             {
984                 result += reader.GetDecimal(0);
985             }
986         }
987     }
988 }
989
990 public static void ExecuteQuery(string sql, out string result)
991 {
992     using (SqlConnection conn = new SqlConnection(connStr))
993     {
994         conn.Open();
995         using (SqlCommand cmd = new SqlCommand(sql, conn))
996         {
997             SqlDataReader reader = cmd.ExecuteReader();
998             result = "";
999             while (reader.Read())
1000             {
1001                 result += reader.GetString(0) + "\n";
1002             }
1003         }
1004     }
1005 }
1006
1007 public static void ExecuteQuery(string sql, out double result)
1008 {
1009     using (SqlConnection conn = new SqlConnection(connStr))
1010     {
1011         conn.Open();
1012         using (SqlCommand cmd = new SqlCommand(sql, conn))
1013         {
1014             SqlDataReader reader = cmd.ExecuteReader();
1015             result = 0;
1016             while (reader.Read())
1017             {
1018                 result += reader.GetDouble(0);
1019             }
1020         }
1021     }
1022 }
1023
1024 public static void ExecuteQuery(string sql, out int result)
1025 {
1026     using (SqlConnection conn = new SqlConnection(connStr))
1027     {
1028         conn.Open();
1029         using (SqlCommand cmd = new SqlCommand(sql, conn))
1030         {
1031             SqlDataReader reader = cmd.ExecuteReader();
1032             result = 0;
1033             while (reader.Read())
1034             {
1035                 result += reader.GetInt32(0);
1036             }
1037         }
1038     }
1039 }
1040
1041 public static void ExecuteQuery(string sql, out decimal result)
1042 {
1043     using (SqlConnection conn = new SqlConnection(connStr))
1044     {
1045         conn.Open();
1046         using (SqlCommand cmd = new SqlCommand(sql, conn))
1047         {
1048             SqlDataReader reader = cmd.ExecuteReader();
1049             result = 0;
1050             while (reader.Read())
1051             {
1052                 result += reader.GetDecimal(0);
1053             }
1054         }
1055     }
1056 }
1057
1058 public static void ExecuteQuery(string sql, out string result)
1059 {
1060     using (SqlConnection conn = new SqlConnection(connStr))
1061     {
1062         conn.Open();
1063         using (SqlCommand cmd = new SqlCommand(sql, conn))
1064         {
1065             SqlDataReader reader = cmd.ExecuteReader();
1066             result = "";
1067             while (reader.Read())
1068             {
1069                 result += reader.GetString(0) + "\n";
1070             }
1071         }
1072     }
1073 }
1074
1075 public static void ExecuteQuery(string sql, out double result)
1076 {
1077     using (SqlConnection conn = new SqlConnection(connStr))
1078     {
1079         conn.Open();
1080         using (SqlCommand cmd = new SqlCommand(sql, conn))
1081         {
1082             SqlDataReader reader = cmd.ExecuteReader();
1083             result = 0;
1084             while (reader.Read())
1085             {
1086                 result += reader.GetDouble(0);
1087             }
1088         }
1089     }
1090 }
1091
1092 public static void ExecuteQuery(string sql, out int result)
1093 {
1094     using (SqlConnection conn = new SqlConnection(connStr))
1095     {
1096         conn.Open();
1097         using (SqlCommand cmd = new SqlCommand(sql, conn))
1098         {
1099             SqlDataReader reader = cmd.ExecuteReader();
1100             result = 0;
1101             while (reader.Read())
1102             {
1103                 result += reader.GetInt32(0);
1104             }
1105         }
1106     }
1107 }
1108
1109 public static void ExecuteQuery(string sql, out decimal result)
1110 {
1111     using (SqlConnection conn = new SqlConnection(connStr))
1112     {
1113         conn.Open();
1114         using (SqlCommand cmd = new SqlCommand(sql, conn))
1115         {
1116             SqlDataReader reader = cmd.ExecuteReader();
1117             result = 0;
1118             while (reader.Read())
1119             {
1120                 result += reader.GetDecimal(0);
1121             }
1122         }
1123     }
1124 }
1125
1126 public static void ExecuteQuery(string sql, out string result)
1127 {
1128     using (SqlConnection conn = new SqlConnection(connStr))
1129     {
1130         conn.Open();
1131         using (SqlCommand cmd = new SqlCommand(sql, conn))
1132         {
1133             SqlDataReader reader = cmd.ExecuteReader();
1134             result = "";
1135             while (reader.Read())
1136             {
1137                 result += reader.GetString(0) + "\n";
1138             }
1139         }
1140     }
1141 }
1142
1143 public static void ExecuteQuery(string sql, out double result)
1144 {
1145     using (SqlConnection conn = new SqlConnection(connStr))
1146     {
1147         conn.Open();
1148         using (SqlCommand cmd = new SqlCommand(sql, conn))
1149         {
1150             SqlDataReader reader = cmd.ExecuteReader();
1151             result = 0;
1152             while (reader.Read())
1153             {
1154                 result += reader.GetDouble(0);
1155             }
1156         }
1157     }
1158 }
1159
1160 public static void ExecuteQuery(string sql, out int result)
1161 {
1162     using (SqlConnection conn = new SqlConnection(connStr))
1163     {
1164         conn.Open();
1165         using (SqlCommand cmd = new SqlCommand(sql, conn))
1166         {
1167             SqlDataReader reader = cmd.ExecuteReader();
1168             result = 0;
1169             while (reader.Read())
1170             {
1171                 result += reader.GetInt32(0);
1172             }
1173         }
1174     }
1175 }
1176
1177 public static void ExecuteQuery(string sql, out decimal result)
1178 {
1179     using (SqlConnection conn = new SqlConnection(connStr))
1180     {
1181         conn.Open();
1182         using (SqlCommand cmd = new SqlCommand(sql, conn))
1183         {
1184             SqlDataReader reader = cmd.ExecuteReader();
1185             result = 0;
1186             while (reader.Read())
1187             {
1188                 result += reader.GetDecimal(0);
1189             }
1190         }
1191     }
1192 }
1193
1194 public static void ExecuteQuery(string sql, out string result)
1195 {
1196     using (SqlConnection conn = new SqlConnection(connStr))
1197     {
1198         conn.Open();
1199         using (SqlCommand cmd = new SqlCommand(sql, conn))
1200         {
1201             SqlDataReader reader = cmd.ExecuteReader();
1202             result = "";
1203             while (reader.Read())
1204             {
1205                 result += reader.GetString(0) + "\n";
1206             }
1207         }
1208     }
1209 }
1210
1211 public static void ExecuteQuery(string sql, out double result)
1212 {
1213     using (SqlConnection conn = new SqlConnection(connStr))
1214     {
1215         conn.Open();
1216         using (SqlCommand cmd = new SqlCommand(sql, conn))
1217         {
1218             SqlDataReader reader = cmd.ExecuteReader();
1219             result = 0;
1220             while (reader.Read())
1221             {
1222                 result += reader.GetDouble(0);
1223             }
1224         }
1225     }
1226 }
1227
1228 public static void ExecuteQuery(string sql, out int result)
1229 {
1230     using (SqlConnection conn = new SqlConnection(connStr))
1231     {
1232         conn.Open();
1233         using (SqlCommand cmd = new SqlCommand(sql, conn))
1234         {
1235             SqlDataReader reader = cmd.ExecuteReader();
1236             result = 0;
1237             while (reader.Read())
1238             {
1239                 result += reader.GetInt32(0);
1240             }
1241         }
1242     }
1243 }
1244
1245 public static void ExecuteQuery(string sql, out decimal result)
1246 {
1247     using (SqlConnection conn = new SqlConnection(connStr))
1248     {
1249         conn.Open();
1250         using (SqlCommand cmd = new SqlCommand(sql, conn))
1251         {
1252             SqlDataReader reader = cmd.ExecuteReader();
1253             result = 0;
1254             while (reader.Read())
1255             {
1256                 result += reader.GetDecimal(0);
1257             }
1258         }
1259     }
1260 }
1261
1262 public static void ExecuteQuery(string sql, out string result)
1263 {
1264     using (SqlConnection conn = new SqlConnection(connStr))
1265     {
1266         conn.Open();
1267         using (SqlCommand cmd = new SqlCommand(sql, conn))
1268         {
1269             SqlDataReader reader = cmd.ExecuteReader();
1270             result = "";
1271             while (reader.Read())
1272             {
1273                 result += reader.GetString(0) + "\n";
1274             }
1275         }
1276     }
1277 }
1278
1279 public static void ExecuteQuery(string sql, out double result)
1280 {
1281     using (SqlConnection conn = new SqlConnection(connStr))
1282     {
1283         conn.Open();
1284         using (SqlCommand cmd = new SqlCommand(sql, conn))
1285         {
1286             SqlDataReader reader = cmd.ExecuteReader();
1287             result = 0;
1288             while (reader.Read())
1289             {
1290                 result += reader.GetDouble(0);
1291             }
1292         }
1293     }
1294 }
1295
1296 public static void ExecuteQuery(string sql, out int result)
1297 {
1298     using (SqlConnection conn = new SqlConnection(connStr))
1299     {
1300         conn.Open();
1301         using (SqlCommand cmd = new SqlCommand(sql, conn))
1302         {
1303             SqlDataReader reader = cmd.ExecuteReader();
1304             result = 0;
1305             while (reader.Read())
1306             {
1307                 result += reader.GetInt32(0);
1308             }
1309         }
1310     }
1311 }
1312
1313 public static void ExecuteQuery(string sql, out decimal result)
1314 {
1315     using (SqlConnection conn = new SqlConnection(connStr))
1316     {
1317         conn.Open();
1318         using (SqlCommand cmd = new SqlCommand(sql, conn))
1319         {
1320             SqlDataReader reader = cmd.ExecuteReader();
1321             result = 0;
1322             while (reader.Read())
1323             {
1324                 result += reader.GetDecimal(0);
1325             }
1326         }
1327     }
1328 }
1329
1330 public static void ExecuteQuery(string sql, out string result)
1331 {
1332     using (SqlConnection conn = new SqlConnection(connStr))
1333     {
1334         conn.Open();
1335         using (SqlCommand cmd = new SqlCommand(sql, conn))
1336         {
1337             SqlDataReader reader = cmd.ExecuteReader();
1338             result = "";
1339             while (reader.Read())
1340             {
1341                 result += reader.GetString(0) + "\n";
1342             }
1343         }
1344     }
1345 }
1346
1347 public static void ExecuteQuery(string sql, out double result)
1348 {
1349     using (SqlConnection conn = new SqlConnection(connStr))
1350     {
1351         conn.Open();
1352         using (SqlCommand cmd = new SqlCommand(sql, conn))
1353         {
1354             SqlDataReader reader = cmd.ExecuteReader();
1355             result = 0;
1356             while (reader.Read())
1357             {
1358                 result += reader.GetDouble(0);
1359             }
1360         }
1361     }
1362 }
1363
1364 public static void ExecuteQuery(string sql, out int result)
1365 {
1366     using (SqlConnection conn = new SqlConnection(connStr))
1367     {
1368        
```



根据仿真结果：

- 表 3-1中的数据不存在系统误差，不存在粗大误差，最终结果为  $3.577 \pm 0.006V$
- 表 3-2中的数据不存在系统误差，不存在粗大误差，最终结果为  $3.581 \pm 0.003V$
- 两次结果进行不等精度处理，最终结果  $3.580 \pm 0.009$

## (二) 电流不等精度数据处理结果

针对电流不等精度数据处理结果如图 6-2所示。

| 调试控制台                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | 终端                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 对数据1进行等精度处理：<br>第一步：求算数平均值<br>求测量值的算数平均值为18.00667<br><br>第二步：求残差<br>各测量值的残差误差为0.00333,0.00333,0.00333,-0.00667,0.00333,0.00333,-0.00667,0.00333,-0.00667<br><br>第三步：校核算数平均值及其残差误差<br>利用规则2进行校核：根据多次测量的校核规则，以上计算正确，校核通过<br><br>第四步：判断系统误差<br>按残余误差校核法： $\Delta/\sigma=0.00667/0.005$ ， $\Delta$ 较大，故判断该测量列无系统误差存在<br>按不同公式计算标准误差： $w/\sigma=18.00667/0.005$ ， $w/\sigma=3.60133$ ，故判断该测量列无系统误差存在<br><br>第五步：求测量列单次测量的标准差<br>根据贝塞尔公式求得： $\sigma=0.005$<br>根据别捷尔公式求得： $\sigma=0.00591$<br><br>第六步：判断粗大误差<br>根据3σ原则（未以符号表示）：无粗大误差<br>根据拉普拉斯准则（ $\sigma=0.005$ ）：最大值不含有粗大误差；最小值不含有粗大误差<br><br>第七步：求算数平均值的标准差<br>求得算数平均值的标准差为0.00167<br><br>第八步：求算数平均值的极限误差<br>共9个测量值，小于10个，应选用t分布计算算数平均值的极限误差，求得算数平均值的极限误差为0.00385<br><br>第九步：写出最后测量结果<br>测量结果为：18.00700-004<br><br>对数据2进行不等精度处理：<br>第一步：求算数平均值<br>求测量值的算数平均值为18.05<br><br>第二步：求残差<br>各测量值的残差误差为-0.05,0.05,0.05,-0.05,0.05,-0.05<br><br>第三步：校核算数平均值及其残差误差<br>利用规则2进行校核：根据多次测量的校核规则，以上计算正确，校核通过<br><br>第四步：判断系统误差<br>按残余误差校核法： $\Delta/\sigma=0.05/0.05$ ， $\Delta$ 较大，故判断该测量列有系统误差存在<br>按不同公式计算标准误差： $w/\sigma=0.253/0.05$ ， $w/\sigma=5.06$ ，故判断该测量列无系统误差存在<br><br>第五步：求测量列单次测量的标准差<br>根据贝塞尔公式求得： $\sigma=0.05477$<br>根据别捷尔公式求得： $\sigma=0.0683$ | 第九步：写出最后测量结果<br>测量结果为：18.00700-004<br><br>对数据2进行等精度处理：<br>第一步：求算数平均值<br>求得测量值的算数平均值为18.05<br><br>第二步：求残差<br>各测量值的残差误差为-0.05,0.05,0.05,-0.05,0.05,-0.05<br><br>第三步：校核算数平均值及其残差误差<br>利用规则2进行校核：根据多次测量的校核规则，以上计算正确，校核通过<br><br>第四步：判断系统误差<br>按残余误差校核法： $\Delta/\sigma=0.05/0.05$ ， $\Delta$ 较大，故判断该测量列有系统误差存在<br>按不同公式计算标准误差： $w/\sigma=0.253/0.05$ ， $w/\sigma=5.06$ ，故判断该测量列无系统误差存在<br><br>第五步：求测量列单次测量的标准差<br>根据贝塞尔公式求得： $\sigma=0.05477$<br>根据别捷尔公式求得： $\sigma=0.0683$<br><br>第六步：判断粗大误差<br>根据3σ原则（未以符号表示）：无粗大误差<br>根据拉普拉斯准则（ $\sigma=0.05$ ）：最大值不含有粗大误差；最小值不含有粗大误差<br><br>第七步：求算数平均值的标准差<br>求得算数平均值的标准差为0.02236<br><br>第八步：求算数平均值的极限误差<br>共6个测量值，小于10个，应选用t分布计算算数平均值的极限误差，求得算数平均值的极限误差为0.05747<br><br>第九步：写出最后测量结果<br>测量结果为：18.0500-00<br><br>对两组数据与精度处理的结果进行不等精度处理<br>第一步：求加权算数平均值<br>加权算数平均值为18.00666283185843<br><br>第二步：求残差<br>各测量值的残差误差为0.00016,-0.03604<br><br>第三步：求加权算数平均值的标准差<br>加权算数平均值的标准差为0.00399<br><br>第四步：求加权算数平均值的极限误差<br>位置系数取k=3，求得加权算数平均值的极限误差为0.01197<br><br>第五步：写出最后测量结果<br>测量结果为：18.00500-02 |

图 6-2: 电流不等精度处理结果

根据仿真结果：

- 表 3-3中的数据不存在系统误差，不存在粗大误差，最终结果为  $18.05 \pm 0.06\mu A$
- 表 3-4中的数据不存在系统误差，不存在粗大误差，最终结果为  $18.087 \pm 0.004$
- 两次结果进行不等精度处理，最终结果  $18.09 \pm 0.02\mu A$

## (三) 最小二乘法估算结果

最小二乘法的估计量结果如图 6-3所示。

| 调试控制台 | 终端                                                 |
|-------|----------------------------------------------------|
|       | 电阻测量结果为：198.83±0.03<br>电流测量结果为：0.01±0.02<br>julia> |

图 6-3: 最小二乘法估算结果

电阻估算结果为:  $198.83 \pm 0.03k\Omega$ ，根据待测电阻实际的色环为：红黑黑橙棕，可知，电阻实际阻值应该为  $200k\Omega \pm 1\% = 200k\Omega \pm 2k\Omega$ 。因此证明测量值正确，最小二乘法估算正确。

## 七、 结论

本文通过误差理论与数据处理的相关手段，对测得电压和电流进行了统计分析，并使用最小二乘法作出了对电阻值的估计。

# 误差理论与数据处理 项目作业报告

游寅龙 韩昱享 黄林烁 陈旭洋

第一组

2024 年 5 月 28 日

## 目录

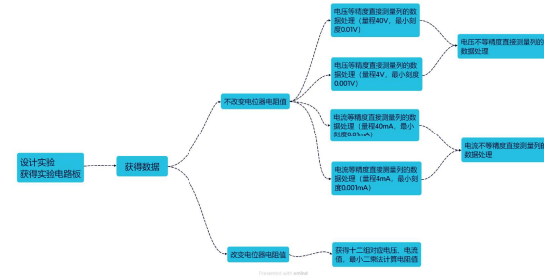
- 1 引言
- 2 研究流程
  - 实验设计
  - 数据测量
- 3 处理步骤
- 4 程序编写
  - 软件环境
  - 完整代码
- 5 结果展示
- 6 项目亮点
- 7 总结

## 引言

本项目为研究误差理论与数据处理知识的应用，围绕物理实验“伏安法测电阻”的原理展开了一系列实验。实验进行了一系列电压和电流的数据测量，并通过误差理论与数据处理的相关手段，对测得电压和电流进行了统计分析，最后使用最小二乘法作出了对电阻值的估计。

## 研究流程

经组内研究探讨，总结出了如下图研究流程。

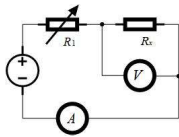


## 实验设计

欧姆定律：

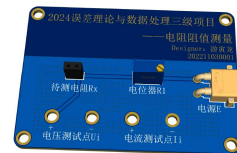
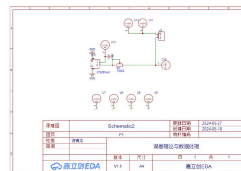
$$R = \frac{U}{I}$$

欧姆定律为“伏安法”测电阻的原理公式。基于此基本公式，设计了右侧的原理电路。



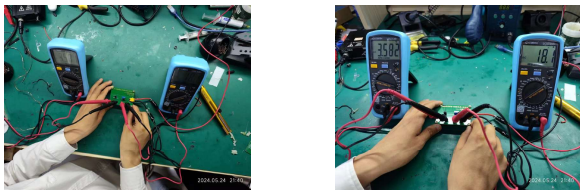
由于我们在实际电路中采用电流表外接的形式。因此需要在后续数据处理时考虑电压表内阻通过的电流，对实验的影响。

基于实验原理图，设计出实际的原理图，电路板仿真图和电路板实物图。



数据测量

在实际测量中，使用了两个同样型号（型号：LCSC530+）的万用表进行。



- 左侧表作为电压表使用，测量电压
- 右侧表作为电流表使用，测量电流

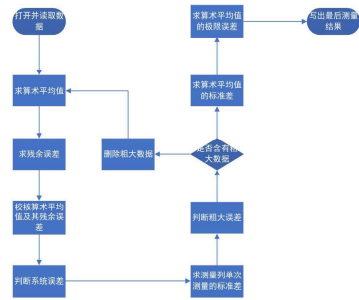
测量结果如下所示

| 项目   | 电压测量  |        | 电流测量  |        | 最小二乘估算 |        |
|------|-------|--------|-------|--------|--------|--------|
|      | 40V   | 4V     | 400μA | 40μA   | 4V     | 40μA   |
| 量程   | 0.01V | 0.001V | 0.1μA | 0.01μA | 0.001V | 0.01μA |
| 最小分度 | 0.01V | 0.001V | 0.1μA | 0.01μA | 0.001V | 0.01μA |
| 测量结果 | 3.57  | 3.578  | 18.0  | 18.09  | 0.530  | 2.83   |
|      | 3.58  | 3.580  | 18.1  | 18.09  | 0.569  | 3.01   |
|      | 3.58  | 3.582  | 18.1  | 18.09  | 1.013  | 5.25   |
|      | 3.58  | 3.580  | 18.0  | 18.08  | 1.822  | 9.33   |
|      | 3.58  | 3.577  | 18.1  | 18.09  | 1.824  | 9.36   |
|      | 3.57  | 3.583  | 18.0  | 18.09  | 2.058  | 10.49  |
|      |       | 3.586  |       | 18.08  | 2.557  | 13.00  |
|      |       | 3.579  |       | 18.09  | 2.701  | 13.73  |
|      |       | 3.584  |       | 18.08  | 3.039  | 15.42  |
|      |       |        |       |        | ...    | ...    |
|      |       |        |       |        |        |        |
|      |       |        |       |        |        |        |
| 组数   | 6     | 9      | 6     | 9      | 16     | 16     |

处理步骤

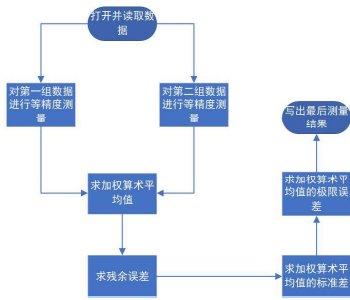
等精度处理步骤：

- 1 求算术平均值
- 2 求残余误差
- 3 校核算术平均值及其残余误差
- 4 判断系统误差
- 5 求测量列单次测量的标准差
- 6 判断粗大误差
- 7 求算术平均值的标准差
- 8 求算术平均值的极限误差
- 9 写出最后测量结果



不等精度处理步骤：

- 1 求加权算术平均值
- 2 求残余误差
- 3 求加权算术平均值的标准差
- 4 求加权算术平均值的极限误差
- 5 写出最后测量结果



最小二乘法（矩阵法）处理步骤：

- 1 列出误差方程
- 2 列出正规方程
- 3 求解估计量
- 4 计算测量数据的标准差
- 5 计算估计量的标准差
- 6 写出最后估算结果

$V = L - A\hat{X}$

$C = A^T A$

$\hat{X} = C^{-1}A^T L = (A^T A)^{-1}A^T L$

$\sigma = \sqrt{\frac{V^T V}{n - t}}$

软件环境

数据处理软件：MWorks  
环境语言：Julia

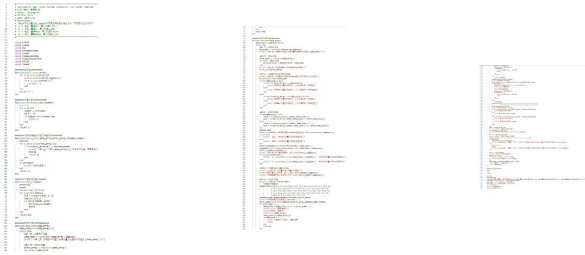


数据管理软件：Excel  
环境语言：VBA



完整代码

共三个文件: 等精度.jl 不等精度.jl 矩阵法最小二乘法.jl  
代码量共计 380+ 行 (等精度 242 行; 不等精度 76 行; 矩阵法最小二乘法 64 行)  
等精度:



不等精度:



最小二乘法估算 (矩阵法):



结果展示

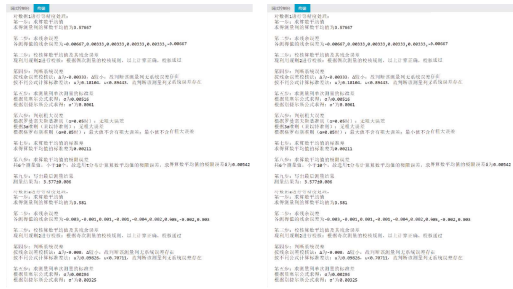
等精度:

- 1 求算术平均值
- 2 求残余误差
- 3 校核算术平均值及其残余误差
- 4 判断系统误差 (两种方法)
- 5 求测量列单次测量的标准差
- 6 判断粗大误差 (三种方法)
- 7 求算术平均值的标准差
- 8 求算术平均值的极限误差
- 9 写出最后测量结果



不等精度:

- 1 求加权算术平均值
- 2 求残余误差
- 3 求加权算术平均值的标准差
- 4 求加权算术平均值的极限误差
- 5 写出最后测量结果



实验亮点

实验完整性

具有完整的流程, 从设计实验、制作实验用具、实践测量、数据处理、数据验证, 完全的覆盖了一个实验的所有基本步骤。

软件使用

除 MWORKS 和 Excel 以外, 还用到了以下几个软件。  
论文和 PPT 使用 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X} 2_{\epsilon}$ 编译生成, 在公式和格式方面具有更优良的表现。  
电路图和流程图使用 visio 和 mwind 绘制, 在图像表达方面具有更好的便捷性。

最小二乘法 (矩阵法) 估算:

- 1 列出误差方程
- 2 列出正规方程
- 3 求解估计量
- 4 计算测量数据的标准差
- 5 计算估计量的标准差
- 6 写出最后估算结果

电阻色环: 红黑黑橙棕  
阻值:  $200\text{k}\Omega \pm 1\% = 200\text{k}\Omega \pm 2\text{k}\Omega$



代码亮点

90% 原创代码

由于 MWOKRS 网上资源较少（只有官方文档和少部分官方资料）且与 MATLAB 不尽相同，因此没有任何前人经验。  
非原创部分：文件申明注释（30 行）引用自反馈三级项目要求；文件导入（12 行）引用自官方例题。

生成结果自动化

对于任意测量列的数据处理，程序能根据测量次数和组数自动生成完整的等精度和不等精度的全部数据处理。  
等精度数据处理中，可以在算出粗大误差后，自动剔除粗大误差，若剩余数据个数  $n' > 3$ ，则从第一步开始重新运算。

多种方法校验

在等精度数据处理中：  
系统误差判断使用了残余误差校核法，不同公式计算标准差比较法两种方法。  
粗大误差判断使用了莱以特准则、罗曼诺夫斯基准则和格罗布斯准则三种方法。  
有效的降低了出错的可能性。

封装等精度数据处理

将等精度数据处理的代码封装成函数，利用 Julia 语言可以跨文件编译的特性，直接在不等精度数据处理中使用。使得代码进一步精炼，自动化程度更高。

置信系数的取法

根据数据处理的特点，程序自动改变置信系数的取值方法：  
等精度数据处理，个数  $n < 10$  时采用学生氏分布；反之则采用正态分布（正态分布使用积分反解方法，夹逼求出高精度的取值，精确到小数点后 4 位）。  
不等精度数据处理，直接采用  $t_\alpha = 3$  进行计算

多处算法实现

在校核算数平均值及其残余误差步骤中，规则二要求得到实际求得的算术平均值  $\bar{x}$  末位数的一个单位  $A$ 。  
在极限误差保留一位有效值后，需要将算术平均值末尾数的单位大小和其统一。  
对粗大误差舍弃后的重新重头计算。  
MWOKRS 没有都直接可采用的函数，因此需要算法来解决这个问题。

总结

在本次实验中，我们组自设实验方案、测量数据。通过误差理论与数据处理的相关手段，对测得电压和电流进行了统计分析，并使用最小二乘法作出了对电阻值的估计。